

부스에 놓인 카드 9장이 실제 코드에서는 어떻게 살아 있나

UMCApp 은 동아리 운영을 하나로 모은 iOS 앱이다. 전시 카드에 적힌 Apple 기술 9가지가 이 앱 어디에서, 어떤 타입으로, 어떤 화면이 되어 돌아가는지를 저장소의 실제 코드와 Apple 공식 문서로 대조해 정리했다. 쓰지 않는 기술은 쓰지 않는다고 적었다.



01
CoreLocation
위치-방향



02
Apple Intelligence
온디바이스 개인 지능



03
CloudKit
iCloud 동기화



04
CoreML
온디바이스 머신러닝



05
Foundation Models
내장 LLM (iOS 26)



06
Liquid Glass
iOS 26 디자인



07
SiriKit
음성-단축어 연동



08
Nearby Interaction
초광대역 정밀 측위



09
WidgetKit
위젯-라이브 액티비티

이 문서를 읽는 법과, 배경이 되는 앱에 대하여.

이 문서가 하는 일

전시 부스의 카드 9장은 각각 Apple 의 iOS 기술 하나를 소개한다. 카드 뒷면은 그 기술이 **일반적으로** 무엇인지 설명한다. 이 문서는 한 걸음 더 들어가서, 그 9가지가 **UMCApp**이라는 **실제로 돌아가는 앱 안에서** 어떤 파일의 어떤 타입이 되었고 어떤 화면으로 나타나는지를 다룬다.

그래서 각 장은 두 개의 목소리로 쓰였다. 하나는 **Apple 공식 문서** — 이 기술이 무엇이라고 Apple 이 말하는가. 다른 하나는 **이 저장소의 코드** — 그래서 우리가 무엇을 만들었는가. 두 목소리는 문서에서 시각적으로 구분해 두었다.

정직하게 적었다. 9가지 중 하나는 이 앱에 들어 있지 않고, 몇 가지는 일부만 쓰인다. 카드에 있으니 역지로 연결짓는 대신 없으면 없다고 적고, 왜 없는지와 불인다면 어디가 자연스러운지를 대신 적었다. 문서에 나오는 파일 경로·줄 번호·코드는 모두 저장소의 실제 파일과 대조해 확인한 것이다.

판정 세 가지

핵심 이 기술이 빠지면 앱의 대표 기능이 성립하지 않는다. 사용자가 직접 마주하는 화면까지 코드가 이어진다. 3가지	부분 적용 실제로 쓰이지만 기능의 일부이거나, 빼대만 깔려 있고 아직 화면까지 닿지 않았다. 한계는 각 장에 적었다. 5가지	미사용 저장소에 코드가 없다. 왜 없는지와, 도입한다면 어느 기능이 가장 자연스러운지를 제안으로 적었다. 1가지
--	---	--

배경 — UMCApp 은 어떤 앱인가

UMC(University MakeUs Challenge) 동아리의 운영 업무를 하나의 iOS 앱으로 모은 클라이언트다. 디스코드·구글 시트·노션으로 흩어져 있던 공지·출석·스터디 관리·커뮤니티를 앱 하나에서 처리해, 부원이 잡무 대신 성장에 집중하게 하는 것이 목표다. 실제 코드에는 홈·공지·활동·커뮤니티·마이페이지 5개 탭과 전자명함 교환, 온디바이스 AI, 원격 킬스위치까지 들어가 있다.

The Ping	공지를 누가 읽었고 누가 안 읽었는지를 앱이 직접 집계하고, 미확인자에게 다시 알린다.
GPS 스마트 출석	모임 장소에 반경 50m 가상 울타리를 치고, 부원의 좌표가 그 안일 때만 출석을 인정한다.
Mobile-First Admin	출석·경고·공지·스터디 운영을 PC 없이 폰에서 처리한다. 화면 수가 가장 많은 영역이다.
전자명함	QR 스캔과 근거리 무선 교환으로 부원끼리 명함을 주고받는다. 인터넷 없이도 동작한다.
온디바이스 AI	공지 요약·일정 자동 분류·대화 요약을 서버로 보내지 않고 기기 안에서 처리한다.

규모

1,268개 Swift 파일
(외부 패키지 제외, 자체 코드만)

23개 Tuist 프로젝트
Core 11 · Feature 9 · 그 외 3

iOS 26.4 배포 타깃
Swift 6.3 · Xcode 26

4개 실행 타깃
앱 · 위젯 · Watch · 테스트

코드가 흐르는 방향

문서에 나오는 타입 이름들이 어느 층에 있는지 알면 각 장이 훨씬 쉽게 읽힌다. UMCApp 은 네 층으로 나뉘고, 의존은 **왼쪽에서 오른쪽으로만** 흐른다 — 화면은 규칙을 알지만 규칙은 화면을 모른다.

View 사용자가 보는 SwiftUI 화면 <code>NoticeDetailView</code>	ViewModel 화면 상태를 들고 있는 객체 <code>@Observable</code>	UseCase 기능의 규칙. 프로토콜로만 노출 <code>ClassifyNoticeUseCase</code>	Repository 서버·기기 저장소·프레임워크 <code>CoreML</code> · <code>CloudKit</code>
---	--	--	--

Apple 프레임워크는 대부분 맨 오른쪽에서만 등장한다. 덕분에 규칙을 테스트할 때 실제 GPS 나 실제 AI 모델 없이 가짜 구현으로 바꿔 끼울 수 있다 — 이 문서에서 반복해 나오는 패턴이다. 조립은 **DIContainer**가 앱 시작 시 한 번에 한다.



세션 장소 반경 50m 안에 있어야만 출석 버튼이 통과되는 GPS 스마트 출석의 심장

Apple 공식 문서 — 이 기술은 무엇인가

CoreLocation 은 기기의 GPS·Wi-Fi·셀룰러 신호를 조합해 현재 위치를 알려주는 Apple 표준 프레임워크다. 핵심 타입인 `CLLocationManager` 는 위치 권한 요청, 위치 갱신 시작/중지, 정확도 설정 같은 위치 관련 동작을 한곳에서 관리하고, 결과는 `CLLocationManagerDelegate` 로 콜백된다. iOS 17 부터 추가된 `CLMonitor` 는 "이 좌표 반경 N미터 원" 같은 조건(`CircularGeographicCondition`)을 등록해 두면 사용자가 그 안에 들어가고 나가는 순간을 async 이벤트 시퀀스로 알려준다 — 흔히 말하는 지오펜싱(geofencing)이다. 좌표 자체(`CLLocationCoordinate2D`)와 두 지점 사이 거리 계산(`CLLocation.distance(from:)`) 도 이 프레임워크가 제공한다.

"A `CLLocationManager` object is the central place to manage your app's location-related behaviors. Use a location-manager object to configure, start, and stop location services."

Core Location

UMCApp — 이 앱은 이걸로 무엇을 하는가

UMC 부원이 하단 탭의 "활동" 화면에서 출석 가능한 세션 카드를 펼치면 `ActivityCompactMapView` 가 나타나고, 그 `.task` 가 `startLocationUpdate()` → `startGeofenceForAttendance()` → `updateAddressForSession()` 순서로 실행된다. 첫 호출인 `LocationManager.startLocationUpdating()` 은 권한이 없으면 `CLLocationManager.requestWhenInUseAuthorization()` 을 띄우는데, 이때 나오는 문구가 `Project.swift` 에 박아 둔 "GPS 기반 스마트 출석 체크를 위해 위치 정보를 사용합니다." 이다.

이어서 `BaseMapViewModel` 이 세션 장소 좌표와 서버 `scheduleId` 를 식별자로 `CLMonitor.CircularGeographicCondition(center:radius:)` 를 반경 50m(`AttendancePolicy.geofenceRadius`)로 등록하고, 같은 반경이 `BaseMapComponent` 의 `MapCircle` 점선 원으로 지도에 그려진다 — 원 안이면 남색(indigo300), 밖이면 빨강(red300). 지도 우하단 상태바는 `isInside(geofenceId:)` 결과로 "위치 인증됨" / "위치 미인증" 을 실시간 표시하고, 그 옆에는 `MKReverseGeocodingRequest` 로 역지오코딩한 세션 장소 주소가 마귀로 흐른다(길게 눌러 복사·애플 지도 열기 가능).

부원이 출석 버튼을 누르면 `ChallengerAttendanceViewModel.attendanceButtonTapped` → `ChallengerAttendanceUseCase.requestGPSAttendance` 로 내려가 권한·현재 좌표·`isInside(geofenceId: scheduleId)` 3단 가드를 통과해야만 서버에 `locationVerified: true` 로 출석이 기록되고, 범위 밖이면 `DomainError.attendanceOutOfRange` 가 "출석 가능 범위를 벗어났습니다." 로 카드에 표시된다. 화면을 벗어나면 `geofenceCleanup()` 이 `stopAllGeofenceMonitoring()` 을 호출해 등록된 지오펜스를 모두 해제한다.

별개로 운영진이 스튜디오/일정을 등록할 때 쓰는 `MapPlacePickerView` 의 "현재 위치로 이동" 버튼은 `MapPickerLocationProvider` 가 `requestLocation()` 으로 좌표를 한 번만 받아 카메라를 옮긴다.

부원이 보는 화면

"활동" 탭 → 출석 체크에서 세션 카드를 펼치면 세션 장소를 중심으로 한 점선 원이 지도에 그려지고, 내가 그 원 안에 있으면 지도 우하단이 "위치 인증됨"(남색), 밖이면 "위치 미인증"(빨강)으로 바뀐다. 원 밖에서 출석 버튼을 누르면 "출석 가능 범위를 벗어났습니다." 가 뜬다. 상태바를 길게 누르면 세션 주소를 클립보드로 복사하거나 애플 지도로 열 수 있고, 일정 등록 화면의 장소 선택 지도에서는 "현재 위치로 이동" 버튼이 내 좌표로 핀을 옮긴다.

코드에서의 위치

파일	심볼	역할
Core/Foundation/Sources/Location/ LocationManager.swift:28	<code>LocationManager</code>	<code>CLLocationManager</code> + <code>CLMonitor</code> 를 감싼 앱 유일의 canonical 위치 매니저 (권한·위치갱신· 지오펜스·역지오코딩)
Features/Activity/Domain/Sources/UseCases/ Implementations/ ChallengerAttendanceUseCase.swift:100	<code>requestGPSAttendance(sessionId:userId:scheduleId:)</code>	권한 → 좌표 보유 → 지오펜스 내부 3단 가드를 통과해야만 서버에 출석을 보낸다
Features/Activity/Presentation/Sources/ViewModels/ Map/BaseMapViewModel.swift:101	<code>BaseMapViewModel.startGeofenceForAttendance()</code>	세션 좌표를 중심으로 <code>scheduleId</code> 를 식별자 삼아 지오펜스를 등록하는 화면 측 진입점
Features/Activity/Data/Sources/Adapters/ LocationManagerAdapter.swift:28	<code>LocationManagerAdapter</code>	Core 의 <code>CLLocationCoordinate2D/GeocodedAddress</code> 를 도메인 <code>Coordinate/Address</code> 로 변환하는 얇은 어댑터
Core/Foundation/Sources/Location/ GeofenceCalculator.swift:23	<code>GeofenceCalculator</code>	델리게이트 없이 단위 테스트 가능하도록 분리한 순수 거리·반경 판정 로직

파일	심볼	역할
Features/Activity/Domain/Sources/Interfaces/LocationProviding.swift:27	LocationProviding	Domain 이 CoreLocation 을 직접 import 하지 않도록 막는 추상화 (Mock 주입 가능)
Features/Activity/Domain/Sources/Models/Attendance/AttendancePolicy.swift:16	AttendancePolicy.geofenceRadius	출석 유효 반경 50m 상수 — 지오펜스 등록과 지도 원 오버레이가 같은 값을 공유
Core/UIComponents/Sources/PlaceSelectView/MapPickerLocationProvider.swift:29	MapPickerLocationProvider	장소 선택 지도의 '현재 위치로 이동' 전용, 단발성 좌표 조회만 하는 별도 CLLocationManager

```
Core/Foundation/Sources/Location/LocationManager.swift L142-161

public func startGeofenceMonitoring(
    at coordinate: CLLocationCoordinate2D,
    identifier: String,
    radius: CLLocationDistance
) async {
    monitoredGeofences[identifier] = MonitoredGeofence(center: coordinate, radius: radius)

    if monitor == nil {
        monitor = await CLMonitor(monitorName)
    }
    if monitorTask == nil {
        startMonitoringEvents()
    }

    let condition = CLMonitor.CircularGeographicCondition(center: coordinate, radius: radius)
    await monitor?.add(condition, identifier: identifier, assuming: .unsatisfied)
    activeGeofenceId = identifier

    checkCurrentLocationInGeofence(center: coordinate, radius: radius)
}

```

지오펜스 등록의 실제. [CLMonitor](#) 를 지연 생성해 하나만 두고, 세션마다 원형 조건을 식별자(scheduleId)와 함께 추가한다. 동시에 같은 중심/반경을 [monitoredGeofences](#) 딕셔너리에 캐시하는데, [CLMonitor](#) 의 식별자 단위 조회 API 가 async 라 동기 판정([isInside\(geofenceId:\)](#))을 호출하려면 이 캐시가 필요하기 때문이다.

```
Features/Activity/Domain/Sources/UseCases/Implementations/ChallengerAttendanceUseCase.swift L105-124

guard locationProvider.isAuthenticated else {
    throw LocationError.notAuthorized
}

guard let coordinate = locationProvider.currentCoordinate else {
    throw LocationError.locationFailed("현재 위치를 가져올 수 없습니다.")
}

guard locationProvider.isInside(geofenceId: scheduleId) else {
    throw DomainError.attendanceOutOfRange
}

let result = try await repository.requestAttendance(
    scheduleId: scheduleId,
    latitude: coordinate.latitude,
    longitude: coordinate.longitude,
    locationVerified: true
)

return result.toAttendance(sessionId: sessionId, userId: userId)

```

출석의 실제 관문. 권한 → 현재 좌표 보유 → "이 일정의 지오펜스 안" 3단 가드를 모두 통과해야만 서버로 출석 요청이 나간다. 판정을 [isInsideAnyGeofence](#) 가 아니라 [isInside\(geofenceId: scheduleId\)](#) 로 하는 것이 핵심 — 여러 활동이 동시에 지오펜스를 등록해도 다른 세션 반경 안에 있다고 출석이 통과되지 않는다.

설계에서의 선택

쓰지 않은 것 · 한계

- GPS 정확도(accuracy)를 전혀 검사하지 않는다. [desiredAccuracy = kCLLocationAccuracyBest](#) 만 요청할 뿐, 델리게이트가 준 [CLLocation.horizontalAccuracy](#) 를 읽는 코드가 레포 어디에도 없다([grep horizontalAccuracy](#) 결과 0건). 실내처럼 오차가 ±100m 인 측위값도 50m 반경 판정에 그대로 쓰여, 건물 밖에 있어도 "위치 인증됨" 이 뜨거나 그 반대가 될 수 있다. iOS 14+ 의 정밀 위치 거부([accuracyAuthorization](#) , 대략적 위치) 상태도 감지하지 않는다.

- Core 는 Feature 타입을 모른다는 규칙 때문에 어댑터를 한 겹 더 뒀다.
`LocationManager` 주석에 "Core 모듈은 Feature 모듈(Coordinate/Address 등)에 의존할 수 없으므로, 이 타입은 `CLLocationCoordinate2D` / `GeocodedAddress` 같은 Feature-독립 타입만 노출합니다"라고 못박혀 있고, 그 경계를 `LocationManagerAdapter` 가 넘나들며 타입만 바꾼다. 대가는 좌표 변환 코드가 한 겹 늘어나는 것이고, 얻는 것은 `LocationProviding` 프로토콜이 CoreLocation 타입을 하나도 노출하지 않는다는 점이다 — 전부 도메인 타입이라 `ChallengerAttendanceUseCase` 는 `MockLocationProvider` 로 출석 규칙을 단위 테스트할 수 있다
(`Features/Activity/Domain/Tests/UseCases/ChallengerAttendanceUseCaseTests.swift:202`). 다만 `ActivityDomain` 모듈 전체가 CoreLocation 프리가 된 것은 아니다 — `AttendancePolicy.swift:9` 와 `Geofence.swift:9` 는 `CLLocationDistance` 타입 때문에 여전히 import 한다.
- "어느 하나라도 안" 대신 식별자 기반 판정을 선택했다. `LocationProviding` 주석은 `isInsideAnyGeofence` 를 "fallback / UI 상태 표시용"으로만 규정하고, 실제 출석 가드에는 `isInside(geofenceId:)` 를 쓰라고 못 박는다. 여러 활동이 동시 진행될 때 옆 세션 반경 안에 있다고 출석이 통과되는 사고를 막기 위해서다. 다만 `isInsideAnyGeofence` 는 마지막 등록분(`activeGeofenceId`) 하나만 추적하는 한계가 코드 주석에 그대로 적혀 있다.
- 지오펠스 식별자는 반드시 서버 `scheduleId` 여야 한다는 계약을 타입 이름으로 강제한다. `BaseMapViewModel.init` 은 파라미터를 `scheduleId` `geofenceId:` 로 받아 호출부가 개념 이름으로 넘기게 하고, 주석에 " `session.info.sessionId.value` 를 넘기면 컴파일은 되지만 출석이 항상 거부됩니다"라고 실패 모드를 박아 뒀다.
`ChallengerAttendanceSessionView.mapViewModel(for:)` 은 `scheduleId` 가 아직 해석되지 않으면 지도를 그리지 않고 `nil` 을 돌려준다.
- `CLMonitor` 가 watchOS 에 없어서 위치 기능을 두 등급으로 쪼갬다.
`UMCFoundation` 이 iOS/watchOS 멀티플랫폼 모듈이라, 지오펠스 관련 API 전부를 `#if !os(watchOS)` 로 감싸고 위치 조회-권한-역지오코딩만 양쪽 공통으로 남겼다(주석 명시). 실제 `UMCWatchApp` 코드에는 현재 위치 사용처가 없으므로 방어적 분기다.

- 백그라운드 지오펠스는 쓰지 않는다. 권한은 `NSLocationWhenInUsageDescription` 하나뿐이고 `requestAlwaysAuthorization · allowsBackgroundLocationUpdates` 호출이 없으며, `UIBackgroundModes` 에도 `location` 이 없다(`remote-notification` 만). `CLMonitor` 의 진입/이탈 이벤트를 앱이 꺼진 상태에서 받아 자동 출석시키는 흐름은 구현돼 있지 않고, 지오펠스는 세션 카드를 보고 있는 동안만 살아 있다.
- 서버 측 위치 검증이 없어 클라이언트 판정을 그대로 믿는다.
`requestAttendance(... locationVerified: true)` 처럼 검증 결과를 앱이 스스로 true 로 채워 보낸다. 좌표도 같이 보내므로 서버가 재검증할 여지는 있으나, 앱 코드만 놓고 보면 위치 스푸핑에 대한 방어는 없다.
- 만들어 두고 아무도 안 쓰는 API 가 남아 있다.
`GeofenceEvent` (entered/exited/failed)와 `LocationManager.geofenceEvent` 는 `CLMonitor` 이벤트를 받아 갱신하지만 소비처가 0건이라 진입/이탈 순간에 알람을 띄우는 기능은 없다.
`getCurrentLocation()` , `stopGeofenceMonitoring(identifier:)` , `LocationError.timeout` 도 외부 호출부가 없다. 또 `MapPickerLocationProvider` 는 `CLLocationManager` 를 두 번째로 만드는 중복 구현이며, 파일에 `// TODO(#1017): UMCFoundation.LocationManager` 이관(PR #1023) 머지 후 이 타입 제거·통합 검토 가 그대로 남아 있다.

Core Location <https://developer.apple.com/documentation/corelocation>
 CLLocationManager <https://developer.apple.com/documentation/corelocation/cllocationmanager>

CLMonitor.CircularGeographicCondition <https://developer.apple.com/documentation/corelocation/clmonitor-2r51v/circulargeographiccondition>
 CLLocationManagerDelegate <https://developer.apple.com/documentation/corelocation/cllocationmanagerdelegate>



Apple Intelligence

부분 적용

온디바이스 개인 지능

기기가 Apple Intelligence를 쓸 수 있을 때만 AI 기능을 열어 주는 관문 역할

Apple 공식 문서 — 이 기술은 무엇인가

Apple Intelligence는 개별 프레임워크가 아니라, 아이폰·아이패드·맥에 내장된 생성형 모델과 그 위에 얹힌 시스템 기능들을 묶어 부르는 우산 이름이다. Apple 공식 문서는 그 구성원으로 Writing Tools(글 교정·재작성), Genmoji(직접 만드는 이모지), Image Playground(이미지 생성), Visual Intelligence(카메라 인식), App Intents를 통한 Siri-Spotlight 연동, 그리고 앱이 직접 모델을 부를 수 있게 해 주는 Foundation Models를 든다. 이 기능들은 모든 기기에서 커지지 않는다 — 기기가 지원해야 하고, 지역이 지원해야 하고, 사용자가 설정에서 켜야 하고, 모델 파일이 내려받아서 있어야 한다. 그래서 Apple은 앱이 모델을 쓰기 전에

`SystemLanguageModel.default.availability` 로 상태를 먼저 확인하고, 쓸 수 없을 때는 다른 UI를 보여 주라고 명시한다.

"Before you use the model, you need to verify its availability. Model availability depends on whether the device and region supports Apple Intelligence."

Apple Intelligence (Technology Overviews)

UMCApp — 이 앱은 이걸로 무엇을 하는가

이 앱에서 Apple Intelligence는 "AI가 뭘 해 주느냐"보다 "이 아이폰에서 AI를 쓸 수 있느냐"를 판정하고 그 결과에 따라 화면을 다르게 그리는 관문으로 나타난다. 판정은 전부 `SystemLanguageModel.default.availability` 한 줄이고, 앱은 이걸 세 가지 서로 다른 폴백으로 번역한다. 첫째, 커뮤니티 채팅방 (`CommunityThreadRoomView`)에서는 `ThreadSummarizing.isAvailable` 이 false면 미읽음 요약 배너(`isSummaryBannerVisible`)가 아예 뜨지 않고 스레드 메뉴의 "대화 요약" 항목도 `canSummarize` 로 비활성이 된다 — 주석에 "쓸 수 없는 기능을 알려 봐야 사용자가 할 수 있는 일이 없다"고 이유가 적혀 있다.

둘째, 스레드 생성·편집 폼의 `ThreadClassificationCard` 는 반대로 카드를 감추지 않고 "자동 분류를 쓸 수 없어요 / 카테고리리와 이모지를 직접 골라 주세요"라는 안내로 바뀌고, "이모지 변경하기" 버튼은 계속 열어 둔다. 셋째, 홈 탭 일정 등록 화면(`ScheduleRegistrationView`)은 툴바의 sparkles 버튼을 `isAIAvailable` 로 아예 렌더링하지 않아 AI 자동완성 시트로 가는 입구가 사라진다. 반면 공지 에디터(`NoticeEditorViewModel+AI`)만은 버튼을 그대로 두고 누른 뒤 "이 기기에서는 Apple Intelligence를 사용할 수 없습니다"라는 `AlertPrompt` 를 띄운다.

그리고 실제로 모델을 부르는 `FoundationModelsThreadClassifier.classify` / `FoundationModelsThreadSummarizer.summarize` 는 화면이 이미 잠갔더라도 세션 생성 직전에 `isAvailable` 을 한 번 더 확인해, 시트를 연 사이 사용자가 설정에서 Apple Intelligence를 꺼 버린 경우까지 막는다.

부원이 보는 화면

Apple Intelligence를 쓸 수 있는 아이폰에서는 커뮤니티 채팅방 상단에 "미읽음 요약" 배너가 뜨고, 스레드 만들기 화면에 "Apple Intelligence · 온디바이스" 배지가 붙은 분류 카드가 보이며, 일정 등록 화면 오른쪽 위에 반짝이(sparkles) 버튼이 생긴다. 지원하지 않는 기기에서는 요약 배너와 반짝이 버튼이 통째로 사라지고, 분류 카드는 "자동 분류를 쓸 수 없어요 — 카테고리리와 이모지를 직접 골라 주세요"로 바뀌며, 공지 작성 화면에서만 AI 버튼이 남아 있다가 누르면 "AI 기능 사용 불가" 팝업이 뜬다. 공지 AI를 쓸 때는 "Apple Intelligence" 제목과 함께 오늘 쓴 토큰 게이지가 나오고, 다 쓰면 "오늘 사용 가능한 토큰을 모두 사용했어요 / 내일 다시 사용할 수 있어요"로 버튼이 잠긴다.

코드에서의 위치

파일	심볼	역할
Features/Community/Data/Sources/AI/FoundationModelsThreadClassifier.swift:26	<code>FoundationModelsThreadClassifier.isAvailable</code>	스레드 자동 분류가 이 기기에서 가능한지를 <code>availability</code> 로 판정하는 유일한 지점
Features/Community/Data/Sources/AI/FoundationModelsThreadSummarizer.swift:25	<code>FoundationModelsThreadSummarizer.isAvailable</code>	대화 요약이 이 기기에서 가능한지를 <code>availability</code> 로 판정
Features/Community/Domain/Sources/Interfaces/ThreadSummarizing.swift:71	<code>ThreadSummarizing.isAvailable</code>	"기기 지원 여부"를 도메인 계약에 넣어 화면이 FoundationModels를 직접 import 하지 않게 만든 경계
Features/Community/Presentation/Sources/ViewModels/CommunityThreadRoomViewModel+Summary.swift:25	<code>isSummaryBannerVisible</code>	미지원 기기에서는 요약 배너를 통째로 감추는 폴백(안내조차 띄우지 않음)
Features/Community/Presentation/Sources/Components/ThreadClassificationCard.swift:225	<code>ThreadClassificationCard.unavailableContent</code>	미지원 기기에서 카드를 감추지 않고 "직접 골라 주세요" 안내로 바뀌 그리는 반대 방향 폴백
Features/Home/Presentation/Sources/Views/Registration/ScheduleRegistrationView.swift:340	<code>isAIAvailable</code>	일정 등록 툴바의 sparkles 버튼 자체를 <code>availability</code> 로 숨긴다
Features/Notice/Presentation/Sources/ViewModels/NoticeEditor/NoticeEditorViewModel+AI.swift:51	<code>requestAIImprovement()</code>	공지 에디터는 버튼을 남겨 두고 누른 뒤에 "AI 기능 사용 불가" Alert 을 띄우는 세 번째 방식

파일	심볼	역할
Features/Notice/Domain/Sources/Models/AITokenDailyUsageRecord.swift:16	AITokenDailyUsageRecord	멤버별 당일 AI 토큰 사용량을 SwiftData(+CloudKit)에 누적하는 사용량 정책 모델

Features/Community/Data/Sources/AI/FoundationModelsThreadClassifier.swift	L24-41
<pre>// MARK: - Computed Property public var isAvailable: Bool { if case .available = SystemLanguageModel.default.availability { return true } return false } // MARK: - Function public func classify(title: String, description: String) async throws -> ThreadClassification { // 화면이 버튼을 잠갔더라도 그 사이에 Apple Intelligence 가 꺼질 수 있다. 마지막 확인은 // 실제로 세션을 만드는 여기서 한 번 더 한다. guard isAvailable else { throw ThreadClassificationError.unavailable } let trimmedDescription = description.trimmingCharacters(in: .whitespacesAndNewlines) guard !trimmedDescription.isEmpty else { throw ThreadClassificationError.emptyInput } let session = LanguageModelSession(instructions: Constants.instructions)</pre>	

가용성 판정을 `isAvailable` 한 곳에 모으고, 화면이 이미 버튼을 잠갔더라도 세션을 만들기 직전에 한 번 더 확인한다. 사용자가 시트를 열어 둔 채 설정에서 Apple Intelligence 를 꺼 버리는 경우를 막는 이중 게이트다.

Features/Notice/Presentation/Sources/ViewModels/NoticeEditor/NoticeEditorViewModel+AI.swift	L50-63
<pre>@MainActor public func requestAIImprovement() { let plainText = richAttributedContent.string.trimmingCharacters(in: .whitespacesAndNewlines) guard !plainText.isEmpty else { return } guard !isAIProcessing else { return } guard case .available = SystemLanguageModel.default.availability else { alertPrompt = AlertPrompt(title: "AI 기능 사용 불가", message: "이 기기에서는 Apple Intelligence를 사용할 수 없습니다. 설정에서 Apple Intelligence를 활성화해주세요.", positiveBtnTitle: "확인") } return }</pre>	

공지 에디터만 다른 폴백을 쓴다 — 버튼을 숨기지 않고 누른 뒤 Alert 을 띄운다. 커뮤니티-홈이 진입점을 감추는 것과 대비되고, 안내 문구가 "설정에서 켜세요"로 고정돼 있어 기기 자체가 지원하지 않는 경우에는 맞지 않는 안내가 나간다.

설계에서의 선택

- 폴백 전략을 기능마다 의도적으로 다르게 잡았고, 이유가 주석에 남아 있다. 요약 배너는 "미지원 기기에서는 폴백 안내조차 띄우지 않고 통째로 감춘다 — 쓸 수 없는 기능을 알려 봐야 사용자가 할 수 있는 일이 없고, 채팅 화면 위 공간만 잡아먹는다" (CommunityThreadRoomViewModel+Summary.swift:23-24). 반대로 분류 카드는 "미지원이어도 카드를 감추지 않는다. 요약 배너와 달리 여기서는 'AI 가 채워 줄 줄 알았는데 아무 일도 안 일어나는' 상황이 되므로, 직접 고르라는 안내를 대신 띄운다" (CommunityThreadCreateViewModel+Classification.swift:24-25). 같은 availability 값이라도 그 화면에서 사용자가 다음에 할 수 있는 일이 있느냐로 갈랐다.
- 가용성 판정을 화면이 직접 하지 않고 도메인 프로토콜에 넣었다. `ThreadSummarizing / ThreadClassifying` 은 `isAvailable` 을 계약에 포함하며, 주석은 "기기 지원 여부까지 이 계약에 포함해야 화면이 FoundationModels 를 직접 import 하지 않고도 진입점(배너)을 감출 수 있다"고 적는다(ThreadSummarizing.swift:63-64). 덕분에 테스트에서 미지원 기기를 대역으로 그대로 재현한다 — 요약은 `StubSummarizer(isAvailable: false)` (CommunityThreadRoomViewModelTests.swift:1639, 1829), 분류는 `StubClassifier(isAvailable: false)` (CommunityThreadCreateViewModelTests.swift:225).

쓰지 않은 것 · 한계

- Apple 이 권장하는 `UnavailableReason` 분기가 없다. 공식 예제는 `.available / .unavailable(.deviceNotEligible)` (기기 자체가 미지원) / `.unavailable(.modelNotReady)` (모델 내려받는 중) / `.unavailable(let other)` 로 갈라 서로 다른 UI 를 보여 주라고 하고, 설정에서 꺼진 경우는 `.unavailable(.appleIntelligenceNotEnabled)` 로 구분할 수 있다. 하지만 이 앱은 12곳 전부 `if case .available / guard case .available` 만 쓰고 나머지를 한 덩어리로 뭉겐다. 그 결과 공지 에디터는 아이폰 15(= `deviceNotEligible`)에서도 "설정에서 Apple Intelligence를 활성화해주세요"라고 안내한다 — 사용자가 따라 할 수 없는 지시다. 모델을 내려받는 중인 경우도 "영구 미지원"과 똑같이 보인다.
- Apple Intelligence 우산 중 이 앱이 실제로 불인 건 Foundation Models 한 갈래뿐이다. Writing Tools(`writingToolsBehavior`), Image Playground(`import ImagePlayground`), Genmoji 렌더링 (`NSAdaptiveImageGlyph`), Visual Intelligence, App Intents 를 통한 Siri-Spotlight 연동은 `grep` 으로 확인한 결과 앱 코드에 한 줄도 없다. 공지 에디터는 리치 텍스트를 다루면서도 Writing Tools 커스터마이즈를 하지 않고 시스템 기본 동작에 맡긴다.

- 프라이버시 표기를 브랜드 표기보다 앞에 뒀다. 요약 시트와 분류 카드 헤더가 "Apple Intelligence" 옆에 "온디바이스" 캡슐 배지를 함께 붙이는데, 주석이 이유를 두 번 적는다 — 요약 시트는 "'온디바이스' 는 대화 내용이 기기 밖으로 나가지 않는다는 뜻이라 채팅 요약에서는 브랜드 표기보다 중요한 정보다" (ThreadSummarySheet.swift:104-105), 분류 카드는 "'온디바이스' 는 입력한 특징이 기기 밖으로 나가지 않는다는 뜻이라 브랜드 표기만큼 중요한 정보다" (ThreadClassificationCard.swift:96-97). 실제로 요약·분류·일정 자동완성·공지 개선 어느 경로에도 서버 호출이 없다.
- Apple Intelligence 우산의 한 갈래인 Genmoji 는 도입이 아니라 차단 대상으로 다뤘다. 스레드 아이콘 칸은 `String` 바인딩 `TextField` 를 써서 시스템에 adaptive image glyph 미지원을 알리고 (CommunityThreadCreateView.swift:23-25), 그래도 새어 들어오는 Genmoji 의 U+FFFC 는 `CommunityThreadCreateRule.normalizedIcon` 이 버린다 (CommunityThreadCreateUseCase.swift:32). 서버가 표준 유니코드 이모지만 받기 때문인데, 실측이 시뮬레이터라 "Genmoji 쪽은 실기기 재확인 필요하다"고 한계까지 적어 뒀다(CommunityThreadCreateView.swift:29-30).
- "AI 사용량" 정책이 실제 정책으로 동작하지 않는다.
`AITokenDailyUsageRecord` 는 당일 누적 토큰을 `SwiftData` 에 쌓지만, 상한은 그날의 할당량이 아니라 `SystemLanguageModel.default.contextSize` (모델 컨텍스트 창 크기)이고, 게이지가 가득 차 버튼이 잠기는 곳은 공지 에디터의 `AIConfirmationOverlay` 하나뿐이다. 커뮤니티 요약·분류는 토큰을 기록조차 하지 않아 같은 계정이 채팅 요약을 아무리 돌려도 공지 게이지에 반영되지 않는다.

함께 맞물리는 기술 `Foundation Models` `CloudKit`

Apple Intelligence (Technology Overviews) <https://developer.apple.com/documentation/technologyoverviews/apple-intelligence>

SystemLanguageModel <https://developer.apple.com/documentation/foundationmodels/systemlanguagemodel>

SystemLanguageModel.Availability <https://developer.apple.com/documentation/foundationmodels/systemlanguagemodel/availability-swift.enum>

SystemLanguageModel.Availability.UnavailableReason <https://developer.apple.com/documentation/foundationmodels/systemlanguagemodel/availability-swift.enum/unavailablereason>



CARD 03 / 09

CloudKit

부분 적용

iCloud 동기화

명함첩·공지 읽음 표시 같은 기기 로컬 기록을 iCloud로 자동 동기화한다.

Apple 공식 문서 — 이 기술은 무엇인가

CloudKit은 Apple이 운영하는 iCloud 서버에 앱 데이터를 저장하고 같은 Apple 계정의 여러 기기 사이에서 맞춰주는 프레임워크다. 데이터베이스는 세 종류로 나뉘는데, private은 그 계정 본인만 볼 수 있고, shared는 초대된 사람끼리, public은 앱 사용자 전체가 본다. SwiftData는 이 CloudKit을 직접 다루지 않고 CoreData의 `NSPersistentCloudKitContainer`를 통해 감싸서, `@Model`로 정의한 타입을 서버의 `CKRecord`로 자동 변환·동기화한다. 그래서 `ModelConfiguration(cloudKitDatabase:)` 한 줄만 켜면 개발자가 `CKRecord`나 `CKDatabase`를 직접 만질 일이 없다.

"Core Data with CloudKit uses a specific record zone in the CloudKit private database, which is accessible only to the current user. / SwiftData uses the `NSPersistentCloudKitContainer` class from Core Data to handle CloudKit synchronization."

CloudKit

UMCApp — 이 앱은 이걸로 무엇을 하는가

UMCApp은 CloudKit API(`CKRecord`, `CKContainer`)를 한 줄도 직접 쓰지 않는다 — `import CloudKit`이 레포 전체에 0건이다. 대신 앱이 켜질 때 `UMCAppApp.makeModelContainer()` (UMCApp/Sources/UMCAppApp.swift:199)가 `NoticeReadRecord`, `AITokenDailyUsageRecord`, `NoticeHistoryData`, `GenerationMappingRecord`, `ReceivedCardRecord` 다섯 개를 하나의 `Schema`로 묶고 `ModelConfiguration(cloudKitDatabase: .automatic)`로 컨테이너를 만든다.

`.automatic`은 entitlements에 적힌 첫 컨테이너를 찾아가므로 실제 목적지는 `iCloud.com.umc.product`의 **private 데이터베이스**(그 Apple 계정 본인만 접근)이고, 동기화 엔진은 SwiftData가 내부적으로 쓰는 CoreData의 `NSPersistentCloudKitContainer`다. 사용자 체감으로는 이렇다 — 부원이 근거리 명함 교환을 끝내면 `ExchangeCardsUseCase` → `SaveReceivedCardUseCase` → `ReceivedCardRepository.save(_:)`가 `ReceivedCardRecord`를 저장하고, 그 행이 iCloud를 타고 넘어가 같은 계정의 아이폰·아이패드·아이폰에 → 「받은 명함」(`ReceivedCardsView`, 2열 그리드)에도 그대로 나타난다.

공지 상세를 열면 `NoticeDetailViewModel.markAsReadIfNeeded()`가 `persistReadStateIfPossible()`를 거쳐 `NoticeReadRecord`를 쓰고, 다른 기기의 공지 목록에서도 그 공지가 이미 읽음으로 보인다. 홈 프로필 조회가 만든 (기수, 기수ID) 매핑(`GenerationMappingRecord`)과 AI 개선 기능의 일일 토큰 사용량(`AITokenDailyUsageRecord`)도 같은 컨테이너에 실려 기기 간에 이어진다. 동기화가 켜져 있다는 사실은 코드 곳곳의 **중복 제거 로직**으로 드러난다 — CloudKit이 `@Attribute(.unique)`를 지원하지 않아 같은 논리적 행이 두 번 생길 수 있으므로, 세 Repository 모두 fetch 후 수동 upsert + 인메모리 dedup을 한다.

부원이 보는 화면

아이폰·아이패드 → 「받은 명함」 명함첩과 공지 탭의 읽음/안읽음 표시가 같은 Apple 계정의 다른 기기(아이폰 ↔ 아이패드)에서도 같은 상태로 보인다. 사용자가 따로 켜거나 로그인할 UI는 없고, iCloud 로그인 상태면 조용히 맞춰진다.

코드에서의 위치

파일	심볼	역할
UMCApp/Sources/UMCAppApp.swift:199	<code>UMCAppApp.makeModelContainer()</code>	앱 전체 SwiftData 스키마를 정의하고 <code>cloudKitDatabase: .automatic</code> 으로 iCloud 동기화를 켜는 유일한 지점 (실패 시 로컬 → 인메모리 3단 폴백)
UMCApp.entitlements:13	<code>com.apple.developer.icloud-container-identifiers / icloud-services</code>	<code>iCloud.com.umc.product</code> 컨테이너와 CloudKit 서비스를 선언 — <code>.automatic</code> 이 여기서 컨테이너를 찾아가고
Features/BusinessCard/Data/Sources/Persistence/ReceivedCardRecord.swift:20	<code>ReceivedCardRecord</code>	받은 명함 <code>@Model</code> . CloudKit 제약(전 필드 기본값·unique 금지)에 맞춰 설계됐다
Features/BusinessCard/Data/Sources/Repository/ReceivedCardRepository.swift:105	<code>ReceivedCardRepository.dedupedRecords()</code>	CloudKit 동기화가 만든 중복 행을 <code>identityKey(memberId 우선, 없으면 cardID)</code> 기준으로 걸러낸다

파일	심볼	역할
Features/Notice/Data/Sources/NoticeReadRecord.swift:26	NoticeReadRecord.memberKey	공지 읽음 기록. 프로덕션 CloudKit 스키마에 Int64 로 굳은 CD_memberId 를 못 바꿔 새 필드로 분리한 흔적
Features/Notice/Data/Sources/Repositories/NoticeReadRepository.swift:41	NoticeReadRepository.markAsRead(noticeId:memberId:)	unique 제약 없이 fetch 후 수동 upsert — CloudKit 호환을 위한 우회
Features/Home/Data/Sources/Repositories/ChallengerGenRepository.swift:76	ChallengerGenRepository.fetchGenGisuIdPairs()	기수 매핑 조회 시 seenGens 집합으로 CloudKit 동기화 중복을 제거
Core/DI/Sources/DIContainer.swift:183	DIContainer.configured(modelContext:)	CloudKit 이 붙은 mainContext 를 DI 컨테이너에 단일 등록해 모든 Repository 가 같은 스토어를 쓰게 한다

```

UMCApp/Sources/UMCAppApp.swift
L199-226

private static func makeModelContainer() -> ModelContainer {
    let schema = Schema([
        NoticeReadRecord.self,
        AITokenDailyUsageRecord.self,
        NoticeHistoryData.self,
        GenerationMappingRecord.self,
        ReceivedCardRecord.self,
    ])

    do {
        let cloudConfiguration = ModelConfiguration(
            schema: schema,
            groupContainer: .none,
            cloudKitDatabase: .automatic
        )
        return try ModelContainer(for: schema, configurations: [cloudConfiguration])
    } catch {
        let cloudReason = error.localizedDescription
        logger.error("SwiftData CloudKit init failed, falling back to local: \(cloudReason)")
    }

    do {
        let localConfiguration = ModelConfiguration(
            schema: schema,
            groupContainer: .none,
            cloudKitDatabase: .none
        )
        return try ModelContainer(for: schema, configurations: [localConfiguration])
    }
}

```

CloudKit 동기화가 커지는 유일한 지점. 다섯 개 @Model 을 한 스키마로 묶고 .automatic 으로 entitlements 의 iCloud.com.umc.product 컨테이너를 찾아간다. iCloud 초기화가 실패하면 앱을 죽이지 않고 로컬 스토어로 내려앉는다.

```

Features/BusinessCard/Data/Sources/Repository/ReceivedCardRepository.swift
L103-127

/// CloudKit 동기화가 만들 수 있는 중복을 걸러낸다.
/// `fetchRecords()`가 교환 시각 내림차순이라 살아남는 것은 가장 최근 교환분이다.
private func dedupedRecords() throws -> [ReceivedCardRecord] {
    var seenKeys = Set<String>()
    return try fetchRecords().filter { seenKeys.insert($0.identityKey).inserted }
}

// MARK: - Identity

private extension ReceivedCardRecord {

    /// 중복 판정 키. `memberId` (cardLink 파싱값)가 정본이고, **없을 때만** cardID로 대체한다.
    ///
    /// 키를 돌려 가르는 이유: v1 페이로드(cardLink="")나 파싱 불가능한 cardLink를 받으면
    /// `MyCard(payload:)`가 memberId를 빈 문자열로 복원한다. 이때
    /// - 빈 문자열을 하나의 키로 댄다면 **서로 다른 사람이 한 명으로 사라진다** (데이터 손실)
    /// - 그렇다고 전부 통과시키면 CloudKit 동기화 중복이 그대로 쌓인다
    ///
    /// cardID는 교환마다 새 UUID라 정체성이 없는 상대와 **재교환**하면 여전히 새 행이 생긴다.
    /// 그런 신원 정보가 없는 이상 원리적으로 막을 수 없다 - 여기서 막는 것은 동기화 중복이다.
    var identityKey: String {
        memberId.isEmpty ? "card:\(cardID)" : "member:\(memberId)"
    }
}

```

CloudKit 이 unique 제약을 지원하지 않아 생기는 중복을 앱 쪽에서 치우는 코드. 명함첩 목록-검색-개수 모두 이 dedup 을 거친다.

설계에서의 선택

- CKRecord 를 직접 다루지 않고 SwiftData + `cloudKitDatabase:` `.automatic` 만 권 선택. 서버 대신 CloudKit 을 고른 이유는 데이터의 성격에 있다 — 명함첩은 `ReceivedCardRepository.swift:15` 주석대로 "서버 명함 API 부재 확정(2026-08-15 조사)으로 로컬 전용"이고, 공지 읽음 여부도 서버 목록 응답에 없어서(`NoticeReadRecord.swift:13`) 기기가 스스로 기억해야 한다. 즉 서버에 둘 자리가 없는 개인 기록이라, 개인 iCloud private DB 에 엮는 게 백엔드를 새로 만드는 것보다 싸다. 대가는 UMC 서버가 이 데이터를 전혀 볼 수 없다는 것 — 통계-운영에 쓸 수 없고, 사용자가 iCloud 를 꺼두면 그냥 기기 로컬로 남는다.
- CloudKit 이 `@Attribute(.unique)` 를 지원하지 않는다는 제약이 세 Repository 의 구조를 결정했다. `ChallengerGenRepository.swift:33`, `NoticeReadRepository.swift:14`, `ReceivedCardRepository.swift:16` 모두 같은 이유로 "fetch 후 수동 upsert" 를 쓴다. `ReceivedCardRepository.swift:44` 주석은 `#Predicate` 로 DB 단계에서 거르지 않고 굳이 전량 fetch 후 인메모리 필터를 쓰는 이유까지 적어둔다 — 술어 통과분에만 dedup 이 걸리면 `fetchAll()` 과 `search()` 의 결과 규칙이 갈리기 때문. 명함첩이 수백 건 규모라 전량 fetch 비용을 감수한 트레이드오프다.
- 한번 프로덕션에 배포한 CloudKit 스키마는 필드 타입을 바꿀 수 없다. `NoticeReadRecord.swift:22-25` 와 `AITokenDailyUsageRecord.swift:22-24` 는 레거시 v2.2.0 이 `memberId: Int` 로 올려 `CD_memberId` 가 Int64 로 굳었다고 적고, 프로젝트 규칙(ID 는 String 통일)을 지키기 위해 **기존 필드를 고치는 대신 memberKey 라는 새 필드를 만들었다**. 낡은 Int 필드는 서버에 남아 있지만 앱은 더 이상 읽지 않는다.
- iCloud 초기화 실패를 치명적 오류로 다루지 않는다. `makeModelContainer()` 는 CloudKit → 로컬 → 인메모리 3단으로 내려가고, 마지막 단계에서만 `fatalError` . iCloud 계정이 없거나 개발 프로비저닝이 어긋난 기기에서도 앱이 뜬다는 뜻이지만, 그 경우 조용히 동기화가 꺼지고 사용자에게 알리는 UI 는 없다 (로그만 남는다).

쓰지 않은 것 · 한계

- CloudKit API 자체를 직접 쓰는 코드가 0줄이다 — `import CloudKit` 이 레포 전체(Derived-Tuist/.build 제외)에서 한 건도 없고, `CKRecord` / `CKContainer` / `CKDatabase` / `CKSyncEngine` / `CKAccountStatus` 도 등장하지 않는다. CloudKit 은 SwiftData 아래에 숨어 동작할 뿐이라 verdict 를 core 가 아닌 partial 로 둔다.
- private 데이터베이스만 쓴다. shared(다른 사람과 공유)-public(전체 공개) 는 전혀 쓰지 않으므로, 명함첩을 남과 공유하거나 앱 사용자 전체가 보는 데이터를 CloudKit 에 두는 기능은 없다. 부원 간 명함 교환은 CloudKit 이 아니라 근거리 통신(MPCTransport)이 담당한다.
- 동기화 상태를 앱이 관찰하지 않는다. iCloud 로그인 여부 확인 (`CKAccountStatus`), 동기화 실패 알림, `NSPersistentCloudKitContainer.eventChangedNotification` 구독이 모두 없다. 사용자는 지금 동기화가 되는 중인지 꺼져 있는지 알 방법이 없고, 앱은 `ModelContainer` 생성 실패만 로그로 남긴다. Apple 문서가 권하는 DEBUG 전용 `initializeCloudKitSchema()` 호출부도 없어, 스키마 배포는 코드 밖 (CloudKit Console)에서 수동으로 관리되고 있다.
- 위젯(UMCAppWidget)과 워치 앱(UMCWatchApp)은 SwiftData 를 전혀 쓰지 않고 위젯 entitlements 에도 iCloud 항목이 없다. 동기화된 데이터는 본체 앱 안에서만 보인다.

CloudKit <https://developer.apple.com/documentation/cloudkit>

Syncing model data across a person's devices (SwiftData) <https://developer.apple.com/documentation/swiftdata/syncing-model-data-across-a-persons-devices>

ModelConfiguration.CloudKitDatabase.automatic <https://developer.apple.com/documentation/swiftdata/modelconfiguration/cloudkitdatabase-swift.struct/automatic>

Mirroring a Core Data store with CloudKit <https://developer.apple.com/documentation/coredata/mirroring-a-core-data-store-with-cloudkit>



CARD 04 / 09

CoreML

부분 적용

온디바이스 머신러닝

일정 제목·푸시 문구를 기기 안에서 분류해 아이콘과 태그를 정한다

Apple 공식 문서 — 이 기술은 무엇인가

Core ML 은 이미 학습이 끝난 머신러닝 모델을 앱 안에 넣고 기기 위에서 바로 추론시키는 Apple 프레임워크다. Xcode 프로젝트에 `.mlmodel` 파일을 넣으면 모델·입력·출력에 해당하는 Swift 클래스(여기서는 `ScheduleListClassifierML`, `ScheduleListClassifierMLInput` 등)를 자동으로 만들어 주고, 빌드할 때 기기용으로 최적화된 `.mlmodelc` 로 컴파일해 앱 번들에 담는다. 실제 예측은 `MLModel.prediction(from:)` 이 맡으며 서버 왕복 없이 단말 안에서 끝난다. 다만 Apple 문서는 하나의 `MLModel` 인스턴스를 동시에 여러 스레드에서 쓰지 말라고 못 박고 있다.

"Use an MLModel instance on one thread or one dispatch queue at a time. Do this by either serializing method calls to the model, or by creating a separate model instance for each thread and dispatch queue."

Core ML

UMCApp — 이 앱은 이걸로 무엇을 하는가

UMC 부원이 홈 탭을 열면 `HomeViewModel` 이 그 달의 일정을 받아온 뒤 `classifySchedules(_generation:)` 에서 일정마다 `ClassifyScheduleUseCase.execute(title:)` 를 병렬로 돌리고, 이 UseCase 가 `ScheduleClassifierRepository.classifyWithML(title:)` → `MLModel.prediction(from:)` 을 호출해 "회의/스터디/해커톤" 같은 라벨을 받아온다. 그 라벨은 `ScheduleIconCategory` 로 바뀌어 `HomeViewModel.category(for:)` 를 통해 `ScheduleListCard` 의 왼쪽 아이콘(예: 해커톤 → `laptopcomputer`, 발표 → `mic.fill`)이 되고, 분류가 끝나기 전에는 아이콘이 뒤늦게 바뀌어 깜빡이지 않도록 `areCategoriesReady(for:)` 로 리스트 렌더를 잠깐 미룬다.

같은 UseCase 를 활동 탭의 `ActivityViewModel.makeSessions(from:)` 도 재사용해 세션 카드 아이콘을 정하고, 일정 등록 화면 (`ScheduleRegistrationViewModel.titleDidChange(to:)`)에서는 제목을 타이핑할 때마다 분류 결과가 태그 칩으로 자동 추천된다. 푸시 알림 쪽은 경로가 다르다 — `AppDelegate.saveNoticeHistory(from:)` 가 수신한 제목과 본문을 이어 붙여 `ClassifyNoticeUseCase` 로 넘기고, `NoticeClassifierML` 이 내놓은 success/error/warning/info 라벨이 `NoticeHistoryData.icon` 으로 SwiftData 에 저장되어 "알림 보관함"(`NoticeAlarmView`)의 카드 아이콘 모양과 색(초록 체크 / 빨강 X / 노랑 느낌표)이 된다.

두 모델 모두 `Features/Home/Data/Sources/MLModels/` 에 들어 있는 Create ML 텍스트 분류기(`com.apple.coreml.model.preview.type = textClassifier`, 2026-02-18 학습)이며 네트워크 요청 없이 기기 안에서만 돌아간다. 모델 로드 실패하면 `isModelLoaded` 가 false 가 되어 전부 키워드 매칭 테이블로 조용히 내려앉기 때문에, 사용자 입장에서 아이콘 정확도만 떨어질 뿐 화면이 깨지지는 않는다.

부원이 보는 화면

홈 탭 캘린더의 일정 카드 왼쪽 아이콘(회의는 사람 둘, 해커톤은 노트북, 오리엔테이션은 화성기)과 활동 탭 세션 카드 아이콘이 제목만 보고 자동으로 정해진다. 일정 등록 화면에서 제목을 타이핑하면 태그가 자동으로 하나 추천되고, 직접 태그를 고르면 그 뒤로는 추천이 덮어쓰지 않는다. 푸시 알림이 오면 "알림 보관함" 화면의 카드가 내용에 따라 초록 체크(성공)/빨강 X(에러)/노랑 느낌표(경고)-파랑 정보 아이콘으로 나뉘어 쌓인다.

코드에서의 위치

파일	심볼	역할
<code>Features/Home/Data/Sources/Repositories/ScheduleClassifierRepository.swift:43</code>	<code>ScheduleClassifierRepository.classifyWithML(title:)</code>	일정 제목을 <code>ScheduleListClassifierMLInput</code> 으로 감싸 <code>model.prediction(from:)</code> 을 호출하고 "label" 문자열을 카테고리 매핑한다
<code>Features/Home/Data/Sources/Repositories/ScheduleClassifierRepository.swift:122</code>	<code>ScheduleClassifierRepository.loadModel()</code>	<code>Bundle.module</code> 에서 컴파일된 <code>.mlmodelc</code> 경로를 찾아 <code>init(contentsOf:)</code> 로 모델을 로드한다 (실패 시 nil → 키워드 분류로 degrade)
<code>Features/Home/Data/Sources/Repositories/NoticeClassifierRepository.swift:29</code>	<code>NoticeClassifierRepository.classifyWithML(text:)</code>	푸시 제목+본문 문자열을 <code>NoticeClassifierML</code> 로 예측해 success/error/warning/info 라벨을 <code>NoticeAlarmType</code> 으로 변환한다

파일	심볼	역할
Features/Home/Domain/Sources/UseCases/Implementations/ClassifyScheduleUseCase.swift:23	<code>ClassifyScheduleUseCase.execute(title:)</code>	캐시 → 강한 규칙 → CoreML → 키워드 순서로 일정 카테고리를 확정하는 분류 파이프라인
Features/Home/Domain/Sources/UseCases/Implementations/ClassifyNoticeUseCase.swift:21	<code>ClassifyNoticeUseCase.execute(title:content:)</code>	제목과 본문을 공백으로 이어 붙여 CoreML 로 먼저 분류하고 실패 시 키워드로 fallback
Features/Home/Presentation/Sources/ViewModels/HomeViewModel.swift:225	<code>HomeViewModel.classifySchedules(_:generation:)</code>	홈 캘린더의 월별 일정들을 TaskGroup 으로 병렬 분류해 scheduleCategories 딕셔너리를 채운다
Features/Home/Presentation/Sources/ViewModels/ScheduleRegistrationViewModel.swift:369	<code>ScheduleRegistrationViewModel.titleDidChange(to:)</code>	일정 등록 화면에서 제목을 입력할 때마다 분류 결과를 태그 추천으로 반영한다 (사용자가 직접 고르면 중단)
UMCApp/Sources/AppDelegate.swift:196	<code>AppDelegate.saveNoticeHistory(from:)</code>	푸시 수신 시 ClassifyNoticeUseCase 로 아이콘 종류를 정해 SwiftData(NoticeHistoryData) 에 저장한다

```

Features/Home/Data/Sources/Repositories/ScheduleClassifierRepository.swift
L43-62

public func classifyWithML(title: String) -> ScheduleIconCategory? {
    guard let model else { return nil }

    do {
        let input = ScheduleListClassifierMLInput(text: title)
        let prediction = try model.prediction(from: input)
        guard let label = prediction.featureValue(for: "label")?.stringValue else {
            return nil
        }
        return Self.mapMLLabelToCategory(label)
    } catch {
        #if DEBUG
        print(
            "[ScheduleClassifierRepository] classifyWithML 예측 실패: "
            + error.localizedDescription
        )
        #endif
        return nil
    }
}

```

Core ML 추론의 실제 호출부. Xcode 가 .mlmodel 로부터 자동 생성한 `ScheduleListClassifierMLInput` 에 일정 제목을 넣고 `MLModel.prediction(from:)` 을 호출한 뒤, 출력 피쳐 "label" 의 문자열을 앱 카테고리로 바꾼다. 예측이 던지는 에러는 nil 로 삼겨 상위에서 키워드 분류로 내려있게 한다.

```

Features/Home/Data/Sources/Repositories/ScheduleClassifierRepository.swift
L117-136

/// CoreML이 생성한 `ScheduleListClassifierML.urlOfModelInThisBundle`은 `Bundle(for:)`로
/// 자기 코드가 정착 링크된 바이너리(앱/테스트 번들)를 찾는데, HomeData는 staticFramework라
/// 컴파일된 `.mlmodelc`가 별도 리소스 번들(`Home_HomeData.bundle`)에 담겨 있어 그 경로로는
/// 찾지 못하고 강제 인래핑에서 크래시한다. Tuist가 생성한 `Bundle.module`로 직접 경로를
/// 찾아 `init(contentsOf:)`로 우회한다.
private static func loadModel() -> MLModel? {
    guard
        let modelURL = Bundle.module.url(
            forResource: "ScheduleListClassifierML",
            withExtension: "mlmodelc"
        )
    else {
        #if DEBUG
        print("[ScheduleClassifierRepository] CoreML 모델 리소스를 찾지 못했습니다.")
        #endif
        return nil
    }

    do {
        return try ScheduleListClassifierML(contentsOf: modelURL).model
    }
}

```

모델이 staticFramework 로 쏴진 Tuist 프로젝트에서 Core ML 자동 생성 코드의 기본 로드 경로(`urlOfModelInThisBundle`)가 왜 통하지 않는지, 그리고 `Bundle.module` 로 어떻게 우회했는지를 주석이 그대로 설명한다. 이 앱에서 Core ML 관련 코드 중 가장 앱 구조에 특화된 부분이다.

- 모듈 분리 때문에 Core ML 표준 로드 경로를 못 쓴다. HomeData 가 staticFramework 라 컴파일된 `.mlmodelc` 가 `Home_HomeData.bundle` 이라는 별도 리소스 번들에 들어가는데, Xcode 자동 생성 코드의 `urlOfModelInThisBundle` 은 `Bundle(for:)` 기준이라 이를 못 찾고 강제 언래핑에서 크래시한다 (ScheduleClassifierRepository.swift:117-121 주석). 그래서 `Bundle.module.url(forResource:withExtension:"mlmodelc")` + `init(contentsOf:)` 로 우회하고, Tuist 쪽에서도 `Features/Home/Project.swift:15-17` 에 `dataResources: ["Data/Sources/MLModels/**"]` 를 명시해 리소스 번들에 넣는다. 이 워크어라운드 가 조용히 깨지는 걸 잡으려고 `Features/Home/Data/Tests/ScheduleClassifierRepositoryTests.swift:56` 에 `@Test("CoreML 모델이 번들 리소스에서 로드되어 실제 예측을 수행한다")` 테스트를 따로 두었다.
- 모델은 항상 보조다. 프로토콜 (`ScheduleClassifierRepositoryProtocol.swift:11-17`)이 `isModelLoaded` / `classifyWithML` / `classifyWithKeywords` 세 가지를 모두 노출하고, 판단은 Domain 의 UseCase 가 한다 — 모델 미로드거나 예측이 nil 이면 키워드 매칭으로 내려앉는다. 게다가 제목에 OT·오타·오리엔테이션·온보딩이 들어가면 모델 예측을 무시하고 무조건 `.orientation` 으로 고정한다 (`ClassifyScheduleUseCase.swift:32-38, 50-54`). 사람이 짠 규칙이 모델보다 위에 있는 구조다. 이 우선 규칙에는 부작용이 있다 — `orientationTokens` 를 소문자 제목에 `contains` 로 부분 일치시키므로, 라틴 문자 "ot" 가 들어간 제목("Notion 정리", "Bot 스터디")도 모델 예측과 키워드를 모두 무시하고 `.orientation` 으로 고정된다.
- 캐시 정책이 두 모델에서 다르다. 일정은 제목을 trim+lowercase+공백 축약으로 정규화한 키로 `UserDefaults` (`ScheduleClassifierCache.v5`)에 결과를 저장해 재추론을 피하고, 앱 업데이트로 폐기된 카테고리(`testing`)는 로드 시 걸러낸다. 반면 공지 분류는 캐시가 아예 없는데, 그 이유가 프로토콜 주석에 적혀 있다 — "푸시 본문은 매번 다른 문자열이라 캐시 hit율이 사실상 0" (`NoticeClassifierRepositoryProtocol.swift:9-10`).
- 동시 추론은 Apple 문서와 어긋나는 선택이다. `HomeViewModel.swift:229-231` 주석은 "CoreML 추론도 동시 호출이 안전하므로 병렬 태스크로 넘겨도 된다"며 하나의 `MLModel` 인스턴스를 `withTaskGroup` 으로 여러 스레드에서 동시에 호출한다. 하지만 Apple 의 `MLModel` 문서는 인스턴스를 한 번에 한 스레드/디스패치 큐에서만 쓰고, 그러려면 호출을 직렬화하거나 스레드마다 인스턴스를 따로 만들라고 명시한다. 순차 처리하면 첫 렌더가 일정 수만큼 밀린다는 이유는 타당하지만, 문서상 보장된 사용법은 아니다.
- `MLModelConfiguration` 이나 `computeUnits` 를 전혀 지정하지 않는다 (레포 전체 grep 결과 0건). Neural Engine/GPU/CPU 배분은 기본값에 맡기고, 비동기 `MLModel.load(contentsOf:configuration:)` 대신 동기 `init(contentsOf:)` 를 Repository 아니셜라이저 안에서 호출하므로 DI 컨테이너가 처음 resolve 하는 시점에 그 스레드가 로드 시간만큼 블로킹된다.
- 일정 모델이 실제로 내놓을 수 있는 라벨은 13개(testing, networking, leadership, workshop, celebration, general, project, orientation, hackathon, review, meeting, presentation, study)인데 `ScheduleIconCategory` 의 `.fee` (회비, DUES)에 해당하는 라벨은 학습되어 있지 않다. `mapMLLabelToCategory` 의 `case "fee", "dues"` 분기는 현재 번들된 모델로는 도달할 수 없고, 회비 아이콘은 사실상 키워드 매칭("회비", "납부" 등)으로만 나온다.
- Create ML 공식 문서는 텍스트 분류기를 `NLModel` 로 감싸 학습 때와 추론 때의 토큰나이제이션을 맞추라고 권하지만, 이 앱은 `NaturalLanguage` 를 import 하지 않고 `MLModel.prediction(from:)` 을 직접 호출한다. 또 Core ML 활용은 텍스트 분류 두 개가 전부이며, 이미지·음성·Vision 연계 모델은 사용하지 않는다.

Core ML <https://developer.apple.com/documentation/coreml>
MLModel <https://developer.apple.com/documentation/coreml/mlmodel>

Integrating a Core ML Model into Your App <https://developer.apple.com/documentation/coreml/integrating-a-core-ml-model-into-your-app>
Creating a text classifier model <https://developer.apple.com/documentation/createml/creating-a-text-classifier-model>



CARD 05 / 09

Foundation Models

핵심

내장 LLM (iOS 26)

기기 안에서 도는 AI가 일정·공지·스레드를 대신 쓰고 요약하고 분류한다.

Apple 공식 문서 — 이 기술은 무엇인가

iOS 26에서 추가된, Apple Intelligence를 떠받치는 온디바이스 대규모 언어 모델(LLM)을 앱에서 직접 부를 수 있게 해 주는 프레임워크다.

`SystemLanguageModel.default` 로 모델을 잡고 `LanguageModelSession` 을 만들어 프롬프트를 던지면 텍스트를 받는데, 서버로 나가지 않고 기기 안에서 처리된다. 핵심 기능은 Guided Generation이다 — `@Generable` / `@Guide` 매크로로 Swift 타입 자체를 "받고 싶은 출력 형태"로 선언하면, 프레임워크가 제약 샘플링(constrained sampling)으로 모델이 그 형식을 벗어나지 못하게 막고 채워진 객체를 돌려준다. 기기·지역·설정에 따라 못 쓸 수 있어 `availability` 확인이 사용 전 필수다.

"Instead of working with raw strings, the framework provides guided generation, which gives strong guarantees that the response is in a format you expect. ... The framework uses constrained sampling when generating output, which defines the rules on what the model can generate."

Foundation Models

UMCApp — 이 앱은 이걸로 무엇을 하는가

이 앱의 AI 기능 네 갈래가 전부 이 프레임워크 위에 올라가 있고, 두 갈래는 Guided Generation, 두 갈래는 평균 스트리밍으로 갈린다. (1) 홈 탭에서 일정 등록 화면을 열고 우측 상단 `sparkles` 버튼(`ScheduleRegistrationView.swift:234` , `isAIAvailable` 이 참일 때만 그려진다)을 누르면 `AIAutofillSheet` 가 뜨고, "다음주 화요일 오후 2시 스터디 모임" 같은 문장이 `ScheduleRegistrationViewModel.requestAIAutofill()` → `session.respond(to:generating: ScheduleDraft.self)` 로 들어가 제목·시작/종료 ISO 시각·하루종일 여부·카테고리·메모·장소 이름 7개 필드가 채워진 `ScheduleDraft` 로 돌아오고, `applyDraft(_)` 가 폼에 꽂는다.

(2) 커뮤니티 스레드 방에 미읽음이 10개 이상 쌓인 채 들어가면 상단에 `ThreadSummaryBanner` 가 뜨고, 탭하면 `ThreadSummarySheet` 가 `requestSummary()` 를 호출 → DI가 꽂아 준 `FoundationModelsThreadSummarizer` 가 미읽음 구간을 `보낸사람: 내용` 대화록으로 만들어 `ThreadSummaryDraft` (bullets/actionItems 두 배열)로 받아 온다. (3) 스레드 만들기·수정 화면의 "Apple Intelligence" 라벨이 붙은 `ThreadClassificationCard` 는 `classify()` → `FoundationModelsThreadClassifier` 가 `@Guide(.anyOf(...))` 로 STUDY/QNA/PROJECT/FREE 중 하나를 강제 선택하게 해 카테고리·이모지·"왜 이렇게 정했는지" 한 문장을 카드와 폼에 동시에 반영한다.

(4) 공지 쪽은 형태가 다르다 — 공지 작성 화면의 AI 도우미와 AI 요약, 공지 상세 우측 상단 요약 버튼 모두 `streamResponse(to:)` 로 평문을 받아 오버레이/시트에 글자가 흘러나오게 그리고, `SystemLanguageModel.default.tokenCount(for: session.transcript)` 로 잔 사용량을 `AITokenDailyUsageRecord` (SwiftData)에 하루 단위로 누적해 게이지로 보여 준다. 도중에 나는 실패는 `ErrorHandler` 가 `LanguageModelSession.GenerationError` 를 붙잡아 "입력이 너무 길니다", "안전 정책에 의해 처리할 수 없는 내용입니다" 같은 한국어 문구로 바꾼다.

부원이 보는 화면

홈 탭 일정 등록 화면 우측 상단 반짝이 버튼 → "AI 자동완성" 시트에 일정을 말하듯 한 줄 적으면 폼이 채워진다(장소는 이름만 채워지고 지도에서 직접 확정해야 등록 버튼이 열린다). 공지 상세 우측 상단 반짝이 버튼 → 요약 시트에 글자가 실시간으로 흘러나온다. 공지 작성 화면 → "AI 도우미"로 본문을 다듬거나, "AI 요약"으로 디스코드 메시지를 붙여넣어 제목+본문 초안을 받고 수락 버튼을 눌러 반영된다(오늘 쓴 토큰 게이지가 함께 보인다). 커뮤니티 스레드 방에 안 읽은 메시지가 10개 넘게 쌓이면 상단 배너 → 시트에서 핵심 요약 불릿과 "내가 할 일" 체크 목록을 준다. 스레드 만들기/수정 화면에는 "Apple Intelligence" 카드가 카테고리·이모지·선택 이유를 채워 주고, 마음에 안 들면 그대로 덮어쓸 수 있다.

코드에서의 위치

파일	심볼	역할
Features/Community/Data/Sources/AI/FoundationModelsThreadSummarizer.swift:17	<code>FoundationModelsThreadSummarizer</code>	스레드 미읽음 대화를 @Generable 구조화 출력으로 요약 (핵심 불릿 + 내 할 일)한다
Features/Community/Data/Sources/AI/FoundationModelsThreadClassifier.swift:18	<code>FoundationModelsThreadClassifier</code>	스레드 제목·특징으로 카테고리/이모지/근거를 지정한다. @Guide(.anyOf)로 후보를 서버 enum 4종에 묶는다
Features/Home/Presentation/Sources/ViewModels/ScheduleDraft.swift:19	<code>ScheduleDraft</code>	자연어 일정 문장에서 뽑아낼 7개 필드를 선언한 @Generable 초안 타입
Features/Home/Presentation/Sources/ViewModels/ScheduleRegistrationViewModel+AI.swift:34	<code>requestAIAutofill()</code>	일정 등록 폼의 AI 자동완성 진입점. respond(to:generating: ScheduleDraft.self) 결과를 폼 필드에 매핑
Features/Notice/Presentation/Sources/ViewModels/NoticeEditor/NoticeEditorViewModel+AI.swift:89	<code>improveContentWithAI()</code>	공지 본문 다듬기. streamResponse 로 글자를 흘리고 tokenCount 로 사용량을 잔다

파일	심볼	역할
Features/Notice/Presentation/Sources/ViewModels/NoticeEditor/NoticeEditorViewModel+AI.swift:279	<code>summarizeWithAI(sourceText:)</code>	디스코드/이메일 등 외부 텍스트 → 공지 제목+본문 초안. 평문 출력을 TITLE:/BODY: 로 직접 파싱
Features/Notice/Presentation/Sources/ViewModels/NoticeDetail/NoticeDetailViewModel+AI.swift:57	<code>performReadingSummary(content:)</code>	공지 상세의 읽기 요약. 스트리밍 결과를 시트에 실시간 표시
Core/Foundation/Sources/Error/Handler/ErrorHandler.swift:219	<code>describeGenerationError(_:)</code>	LanguageModelSession.GenerationError 9종을 한국어 사용자 문구로 변환하는 전역 분기

```

Features/Community/Data/Sources/AI/FoundationModelsThreadClassifier.swift
L98-119

@Generable(description: "동아리 채팅 스레드의 카테고리 분류 결과")
private struct ThreadClassificationDraft {

    // MARK: - Property

    @Guide(
        description: """
        스레드 성격에 가장 가까운 카테고리 코드입니다.
        STUDY 는 함께 공부하는 모임, QNA 는 질문과 답변, PROJECT 는 결과물을 만드는 협업,
        FREE 는 나머지 잡담과 친목입니다. 애매하면 FREE 를 고르세요.
        """,
        .anyOf(CommunityThreadCategory.allCases.map(\.rawValue))
    )
    var category: String

    @Guide(
        description: """
        스레드를 한눈에 알아볼 수 있는 이미지 문자 하나입니다.
        설명이나 이름이 아니라 이미지 글자 자체만 쓰고, 두 개 이상 붙이지 마세요.
        """"
    )
    var icon: String

```

Guided Generation 의 핵심. 모델이 자유 문장 대신 이 Swift 구조체를 채우게 하고, `.anyOf` 로 카테고리 후보를 서버 enum 4종(STUDY/QNA/PROJECT/FREE)에 아예 묶어 버려 앱이 문자열을 파싱할 일 자체를 없앤다.

```

Features/Home/Presentation/Sources/ViewModels/ScheduleRegistrationViewModel+AI.swift
L39-57

guard case .available = SystemLanguageModel.default.availability else {
    aiAutofillState = .failed(
        .unknown(message: "이 기기에서는 Apple Intelligence를 사용할 수 없습니다.")
    )
    return
}

aiAutofillState = .loading

do {
    let session = LanguageModelSession { Constants.instructions }
    let response = try await session.respond(to: rawText, generating: ScheduleDraft.self)
    applyDraft(response.content)
    await persistDailyTokenUsage(session: session)
    aiAutofillState = .loaded(true)
} catch {
    // 시트를 띄운 채 문장을 고쳐 바로 재시도할 수 있도록 전역 Alert 대신 인라인으로 알린다.
    aiAutofillState = .failed(.unknown(message: error.localizedDescription))
}

```

일정 자동완성의 실제 호출부. `availability` 확인 → `LanguageModelSession(instructions:)` 생성 → `respond(to:generating:)` 로 `ScheduleDraft` 를 통째로 받아 폼에 반영하는 3단계가 그대로 보인다. 실패는 전역 Alert 이 아니라 시트 안 인라인으로 띄운다.

설계에서의 선택

- Domain 레이어는 FoundationModels 를 절대 import 하지 않는다.
`ThreadSummarizing / ThreadClassifying` 프로토콜 (Features/Community/Domain/Sources/Interfaces/)에 결과 타입뿐 아니라 `isAvailable` 까지 넣어 두어, 화면이 프레임워크를 모르는 채로도 요약 배너를 감추거나 '다시 분류하기' 버튼을 잠글 수 있다. 주석에 명시: "기기 지원 여부까지 이 계약에 포함해야 화면이 FoundationModels 를 직접 import 하지 않고도 진입점 (배너)을 감출 수 있다"(ThreadSummarizing.swift:62-64). 매크로가 붙은 초안 타입(`ThreadSummaryDraft` , `ThreadClassificationDraft`)은 Data 모듈 안 `private` 으로 격리했다.

쓰지 않은 것 · 한계

- 프레임워크의 절반 이상을 안 쓴다. `Tool` 프로토콜(모델이 앱 함수를 호출하는 tool calling), `GenerationOptions` (temperature:최대 토큰), `SystemLanguageModel.Adapter` , 커스텀 `Guardrails` , `prewarm()` , `DynamicGenerationSchema` , 멀티모달(이미지) 프롬프트는 전 코드베이스에 grep 결과 0건이다. 이 앱이 쓰는 건 세션 생성 + `respond/streamResponse` + `@Generable/@Guide` + `availability/contextSize/tokenCount` 까지다.
- 스트리밍과 구조화 출력을 동시에 쓰는 `streamResponse(to:generating:)` (부분 완성 타입) 은 쓰지 않는다. 그래서 Guided Generation 경로(일정 자동완성·스레드 요약·스레드 분류)에서는 결과가 다 나올 때까지 사용자가 스�켈레톤만 보고 기다린다. 온디바이스 모델은 첫 토큰까지 수 초가 걸릴 수 있어 체감 대기가 짧지 않다.

- 세션을 재사용하지 않고 실행 한 번마다 새로 만든다. 이유가 주석에 있다 — 요약기는 "재사용하면 앞 대화가 transcript 에 남아 다음 요약의 컨텍스트를 감아먹고, 전혀 다른 스레드의 내용이 섞여 나올 수 있다"(:15-16), 분류기는 "'다시 분류하기' 가 직전 결과를 되풀이한다"(:16-17). DI 도 그래서 싱글턴을 강제하지 않는다 — "둘 다 실행 한 번마다 세션을 새로 만들고 인스턴스에는 상태가 없다 — 실시간 클라이언트와 달리 단일 인스턴스를 강제할 이유가 없다" (DIContainer+Community.swift:103-104).
- 온디바이스 모델의 좁은 컨텍스트 창을 이중으로 막는다. 요약 입력은 최근 메시지 60개로 자르고 그 위에 4,000자 상한을 한 번 더 건다. 주석: "줄 수를 잘라도 창문 몇 줄이면 컨텍스트를 넘긴다. 문자 수로 한 번 더 막는다. 최신 대화가 더 중요하므로 앞(오래된 쪽)을 버린다"(FoundationModelsThreadSummarizer.swift:67-69, 143-150).
- AI 가 좌표를 만들지 못하게 설계로 막았다. `ScheduleDraft` 는 `placeName` 만 받고 좌표 필드가 아예 없으며, 적용부는 위경도를 (0, 0) 으로 남겨 `isValidPlace` 를 거짓으로 유지해 제출 버튼을 잠근다 — "사용자가 지도에서 장소를 확정해야 (0, 0) 기준 지오펜스가 만들어지지 않는다" (ScheduleRegistrationViewModel+AI.swift:115-119). 환각 위험이 큰 값은 애초에 모델에게 말하지 않는 쪽을 택했다.

- Data 레이어 구현체(`FoundationModelsThreadSummarizer` / `FoundationModelsThreadClassifier`)에 단위 테스트가 없다. Presentation 테스트는 전부 `StubSummarizer` / `StubClassifier` 대역을 꽂아 ViewModel 상태 전이만 검증하므로(CommunityThreadRoomViewModelTests.swift:273, CommunityThreadCreateViewModelTests.swift:40), 프롬프트나 `@Guide` 문구를 고쳤을 때 품질이 나빠져도 CI 가 잡아 주지 않는다. 또한 `availability` 의 실패 사유(.deviceNotEligible / .modelNotReady)를 구분하지 않고 전부 같은 한 문장으로 뭉갠다 — Apple 문서 예시는 사유별 분기를 권한다.

Foundation Models <https://developer.apple.com/documentation/foundationmodels>

Generating Swift data structures with guided generation <https://developer.apple.com/documentation/foundationmodels/generating-swift-data-structures-with-guided-generation>

SystemLanguageModel <https://developer.apple.com/documentation/foundationmodels/systemlanguagemodel>

LanguageModelSession <https://developer.apple.com/documentation/foundationmodels/languagemodelsession>



CARD 06 / 09

Liquid Glass

핵심

iOS 26 디자인

앱의 거의 모든 카드·버튼·툴바 표면을 그리는 기본 재질이다.

Apple 공식 문서 — 이 기술은 무엇인가

Liquid Glass 는 iOS 26 이 도입한 동적 재질(material)로, 유리의 광학적 성질과 유동성을 합친 것이다. 뒤에 있는 콘텐츠를 흐리게(blur) 하고 주변의 색과 빛을 반사하며, 터치·포인터 입력에 실시간으로 반응한다. 툴바·탭바·시트 같은 표준 컴포넌트에는 자동으로 적용되고, 직접 만든 뷰에는 SwiftUI 의 `glassEffect(_:in:)` modifier 로 붙인다. 여러 유리 표면을 `GlassEffectContainer` 로 묶으면 서로의 모양이 섞이거나(blend) 전환 중에 하나에서 다른 하나로 녹아들 듯 변형(morph) 될 수 있다.

"Liquid Glass is a material that blurs content behind it, reflects color and light of surrounding content, and reacts to touch and pointer interactions in real time. Standard components in SwiftUI use Liquid Glass."

Applying Liquid Glass to custom views

UMCApp — 이 앱은 이걸로 무엇을 하는가

UMC 부원이 앱을 켜면 `RootTabView` 가 5개 탭을 `TabView` 로 세우는데, iOS 26 이 탭바와 내비게이션 바를 자동으로 유리 재질로 그리고 `.tabBarMinimizeBehavior(.onScrollDown)` 이 붙어 있어 스크롤을 내리면 탭바가 작게 접힌다. 그 위에 `.tabViewBottomAccessory` 로 `RootTabAccessoryView` 가 유리 캡슐로 떠서, 탭마다 다른 액션(공지 작성, 스레드 생성, 운영진↔챌린저 모드 토글)을 하나만 보여준다. 홈 탭으로 들어가면 `HomeView` 의 일정 카드·최근 공지 카드·상벌점 카드가 각각 `.glassEffect(.regular, in: .rect(corners: .concentric(...)))` 로 그려져 배경 위에 떠 있는 유리판처럼 보이고, 데이터가 없을 때 뜨는 `ContentUnavailableView` 나 로딩 `ProgressView` 도 같은 유리 판으로 자리를 채운다.

활동 탭의 GPS 출석에서는 `ChallengerAttendanceView.attendanceActionView` 가 `GlassEffectContainer` 안에 출석 버튼과 `ChallengerPendingApprovalView` 를 같은 `glassEffectID (attendanceNamespace)` 로 두어, 부원이 출석 버튼을 누르면 버튼 유리가 "승인 대기" 카드 유리로 모양이 녹듯 바뀐다. 운영진 모드로 바꾸면 `OperatorStudyGroupSection` 이 `GlassEffectContainer(spacing: 16)` 으로 `StudyGroupCard` 들을 묶어 카드끼리 유리가 섞이게 하고, 공지 작성 화면에서는 `NoticeEditorAttachmentToolbar` 가 첨부 버튼 줄 전체를 하나의 유리 바로 감싼다.

이 모든 호출은 공용 modifier 를 거치지 않고 각 View 파일이 `.glassEffect` 를 직접 부르는 방식이며, 규약은 코드가 아니라 `docs/claude/design-system.md` 의 variant 선택 표가 잡고 있다.

부원이 보는 화면

활동 탭에서 GPS 출석 버튼을 누르면 버튼 유리가 "승인 대기" 카드로 형태가 녹듯 바뀐다. 홈·공지·스터디 화면의 카드들은 배경을 흐리고 굴절시키는 반투명 유리판으로 떠 있고, 필터 칩은 누르면 손가락을 따라 유리가 놀린다. 탭바는 스크롤을 내리면 작게 접히고, 그 위 유리 캡슐에 탭별 액션 버튼이 하나 뜬다.

코드에서의 위치

파일	심볼	역할
UMCApp/Sources/RootTab/RootTabView.swift:47	<code>RootTabView.body</code>	5개 탭 셀. 시스템이 유리로 그리는 탭바를 스크롤 시 접고(<code>tabBarMinimizeBehavior</code>), 그 위에 유리 캡슐 액세서리를 띄운다
Core/UIComponents/Sources/MainButton/MainButton.swift:30	<code>MainButton</code>	앱 공통 CTA 버튼. 자체적으로 유리를 그리지 않고, Apple 기본 <code>.glass/.glassProminent</code> <code>ButtonStyle</code> 을 호출부가 붙이도록 문서 주석(30행) 이 안내한다
Core/UIComponents/Sources/Badge/InfoBadge.swift:32	<code>InfoBadge.GlassVariant</code>	유리 variant(<code>.regular/.clear</code>)와 tint 를 파라미터로 받는 유일한 공용 래퍼 컴포넌트
Features/Activity/Presentation/Sources/Components/Challenger/Attendance/ChallengerAttendanceView.swift:262	<code>attendanceActionView</code>	GPS 출석 버튼 ↔ 승인 대기 카드를 <code>GlassEffectContainer + glassEffectID</code> 로 모핑 전환
Features/Activity/Presentation/Sources/Components/Operator/Study/StudyGroupCard.swift:138	<code>StudyGroupCard.body</code>	운영진 스터디 그룹 카드. 카드 뷰 자체에 <code>.glassEffect(_:in:)</code> 을 붙이는 Apple 권장 패턴 + 컨테이너 코너 상속 이슈 회피
Features/Activity/Presentation/Sources/Views/Operator/OperatorAttendanceDetailView.swift:373	<code>participantList</code>	List 대신 <code>LazyVStack</code> 을 쓰는 이유를 주석으로 남긴 곳 — List 는 <code>GlassEffectContainer</code> 와 호환되지 않는다

파일	심볼	역할
Core/UIComponents/Sources/Utilities/KeyboardToolbarModifier.swift:29	KeyboardToolbarModifier.bottomToolBar	폼 입력 화면 공통 키보드 톨바. 원형 유리 버튼 3개를 컨테이너로 묶고 glassEffectID 부여
Project.swift:30	infoPlist UIInterfaceStyle	앱을 Light 로 고정. 검증 안 된 다크 유리를 내보내지 않기 위한 결정이 주석으로 명시돼 있다

Features/Activity/Presentation/Sources/Components/Challenger/Attendance/ChallengerAttendanceView.swift	L259-275
<pre> /// 출석 상태에 따른 액션 뷰 (버튼 또는 승인 대기 카드) @ViewBuilder private var attendanceActionView: some View { GlassEffectContainer { switch session.attendanceStatus { case .pendingApproval: // 승인 대기 카드 (버튼에서 모핑 전환) ChallengerPendingApprovalView() .glassEffect(.regular, in: .rect(corners: .concentric(minimum: DefaultConstant.concentricRadius), isUniform: true)) .glassEffectID(Constants.attendanceActionId, in: attendanceNamespace) .transition(actionTransition) // ... (case .beforeAttendance / .present, .late, .absent 생략) } } } </pre>	

GPS 출석의 핵심 전환. 출석 버튼과 승인 대기 카드를 같은 GlassEffectContainer 안에서 같은 glassEffectID 로 묶어, 상태가 바뀌면 유리 모양이 하나에서 다른 하나로 모핑된다. 앱 전체에서 glassEffectID 를 쓰는 3곳 중 하나다.

Features/Activity/Presentation/Sources/Components/Operator/Study/StudyGroupCard.swift	L136-149
<pre> // Apple 가이드 권장 패턴: View 자체에 `.glassEffect(_in:)` 적용 // → GlassEffectContainer 내부 morphing/blending이 정상 동작 .glassEffect(.regular, in: .rect(cornerRadius: Constants.cardCornerRadius)) // 카드 외곽 라운드로 콘텐츠를 한 번 더 clip - 내부 섹션 clip과 더불어 어떤 자식도 // 카드 모서리를 넘지 않도록 보장한다. .clipShape(.rect(cornerRadius: Constants.cardCornerRadius)) // 카드 전체를 펼침/접힘 탭 영역으로 만든다. 내부 Button/Menu(알정 등록·설정·추가)는 // 자체적으로 탭을 소비하므로 토글되지 않고, 빈 영역·타이틀·메타·파트장/멤버 행을 // 탭하면 펼침/접힘이 토글된다. .contentShape(.rect(cornerRadius: Constants.cardCornerRadius)) .onTapGesture { toggleExpansion() } </pre>	

운영진 스티디 그룹 카드. GlassEffectContainer 안에서는 ConcentricRectangle 의 코너 상속이 끊긴다는 걸 알고 고정 24pt 라운드(Constants.cardCornerRadius, 87-93행 주석에 이유 기록)로 못 박았다. 유리 modifier 를 뷰 자체에 붙여야 컨테이너 안 morphing 이 동작한다는 점도 주석에 남아 있다.

설계에서의 선택

- 공공 modifier 로 감싸지 않고 화면마다 직접 부른다. Core/DesignSystem 안에는 glass 라는 문자열이 한 줄도 없고(grep 0건), Core/UIComponents 의 InfoBadge·ChipButton·LoadingView·AttendanceStatusBadge 같은 개별 컴포넌트가 각자 .glassEffect 를 호출한다. MainButton 은 유리를 직접 붙이지 않고 Apple 기본 .glass / .glassProminent ButtonStyle 을 호출부가 적용하는 구조다. 일관성은 코드가 아니라 docs/claude/design-system.md 의 "Glass Effect 선택" 표(.regular=일반 카드, .regular.interactive()=탭 가능 요소, .clear=미디어 위, .glassProminent=Primary 버튼)가 문서로만 강제한다. 트레이드오프는 명확하다 — 새 화면이 표를 어겨도 컴파일러가 잡아주지 않고 리뷰에 의존한다.
- GlassEffectContainer 를 무조건 쓰지 않는다. 오히려 OperatorAttendanceView.swift:409 와 OperatorAttendanceDetailView.swift:163 은 "주의: GlassEffectContainer 로 감싸지 않는다" 를 주석으로 명시한다 — 카드가 ConcentricRectangle (적응형 코너)을 쓰는데 컨테이너 안에서는 코너 상속이 끊겨 .interactive() 놀림 효과가 카드와 다른 크기로 morphing 되기 때문이다. 실제로 .glassEffect 호출은 95곳인데 프로덕션 컨테이너는 7곳뿐이다(8곳 중 1곳은 #Preview 안). 레포 문서 docs/claude/design-system.md:103 은 " GlassEffectContainer 로 그룹화 필수 (오프스크린 렌더링 66% 감소)" 라고 적혀 있어, 문서와 코드가 완전히 일치하지는 않는다.

쓰지 않은 것 · 한계

- import LiquidGlass 는 코드베이스에 0건이다. Liquid Glass 는 별도 프레임워크가 아니라 SwiftUI(그리고 UIKit)에 들어 있는 API 집합이라 import SwiftUI 만으로 쓴다. "Liquid Glass 프레임워크를 붙였다" 는 표현은 정확하지 않다.
- Apple 이 제공하는 유리 API 중 상당수를 안 쓴다. glassEffectUnion , glassEffectTransition , scrollEdgeEffect 는 코드에 한 건도 없고, backgroundExtensionEffect 는 공지 이미지 뷰어 1곳 (NoticeImageCard.swift:151), ToolbarSpacer 는 커뮤니티 목록 1곳 (CommunityThreadListView.swift:71)뿐이다. 사실상 .glassEffect + .glass / .glassProminent ButtonStyle 두 가지에 집중돼 있다.
- 유리 관련 테스트가 없고, 위젯-위치 타킷에도 사용처가 없다. UMCAppWidget / UMCWatchApp 에서 glass 문자열 검색 결과는 0건이다. 렌더링 재질이라 단위 테스트로 검증하기 어려운 특성상 자연스럽지만, "컨테이너 밖 유리를 남발하면 성능이 떨어진다" 는 Apple 경고에 대한 자동 방어선은 없는 셈이다. 대신 ChromaticLens (Metal 셰이더)라는 자체 굴절 효과를 따로 만들어 두었는데, 이걸 Apple 유리가 노출하지 않는 굴절·색분산 강도를 직접 조정하려고 만든 별개 구현이다.

- 모핑 애니메이션은 시도했다가 되돌린 이력이 코드에 남아 있다.
[StudyGroupCard.swift:273-277](#) 주석: collapsed(원형)/expanded(캡슐)
두 뷰를 `glassEffectID` 로 morphing 했더니 전환 중 두 상태가 cross-fade
되며 캘린더 아이콘이 두 개로 겹쳐 보이는 잔상이 생겼고, 결국 아이콘이 항상
존재하는 단일 뷰 + 항상 `.capsule` 배경으로 바꿔 너비·틴트만 변화게 했다.
[docs/claude/design-system.md:104](#) 도 " `glassEffectID` 는 모핑
애니메이션 필요 시만 사용 (CPU 부하)" 로 제한을 건다. 실제 `glassEffectID`
사용은 3곳뿐이다.
- 다크 모드를 포기하는 대신 유리 검증 부담을 줄였다. [Project.swift:30](#) 이
`UIUserInterfaceStyle: Light` 를 박아두고, 주석에 "컬러 토큰 46개에 다크
값이 다 있는데도 출시본이 라이트로 고정해 나갔다... 시안에도 다크 프레임이
없어서, 검증 안 된 다크를 내보내지 않는다" 고 이유를 적어놨다. 유리는 뒤 콘텐츠와
주변 빛을 반사하므로 라이트/다크에서 대비가 크게 달라지는데, 한쪽만 보증하는
선택이다. 또 배포 타킷이 iOS 26.4 라 `if #available` 분기 없이 glass API 를
그대로 쓴다.

함께 맞물리는 기술

[CoreLocation](#)

[Foundation Models](#)

[Nearby Interaction](#)

Applying Liquid Glass to custom views <https://developer.apple.com/documentation/swiftui/applying-liquid-glass-to-custom-views>
`glassEffect(_in:)` [https://developer.apple.com/documentation/swiftui/view/glasseffect\(_:in:\)](https://developer.apple.com/documentation/swiftui/view/glasseffect(_:in:))

`GlassEffectContainer` <https://developer.apple.com/documentation/swiftui/glasseffectcontainer>
`glassEffectID(_in:)` [https://developer.apple.com/documentation/swiftui/view/glasseffectid\(_:in:\)](https://developer.apple.com/documentation/swiftui/view/glasseffectid(_:in:))



CARD 07 / 09

SiriKit

미사용

음성·단축어 연동

쓰지 않는다 — Siri·단축어 진입점이 코드·entitlement 양쪽 모두 0건이다.

Apple 공식 문서 — 이 기술은 무엇인가

SiriKit 은 앱의 기능을 시스템(Siri 음성, 제안, 단축어 앱)에 내어주는 애플의 프레임워크 모음이다. 공식 문서는 이 프레임워크군을 "Empower users to interact with their devices through voice, intelligent suggestions, and personalized workflows" 라고 소개하며, 실제 구현은 [Intents](#) 프레임워크의 [INIntent](#) 서브클래스와 그것을 처리하는 Intents App Extension 으로 이뤄진다. 다만 현재 애플이 권장하는 경로는 SiriKit 이 아니라 2022년에 나온 **App Intents** 프레임워크다 — Swift 로 [AppIntent](#) 프로토콜을 채택한 타입 하나가 곧 "앱이 할 수 있는 동작 하나"가 되고, 그게 Siri·Spotlight·단축어·위젯 설정·컨트를 센터에 동시에 노출된다.

기존 SiriKit 커스텀 인텐트를 쓰던 앱을 위해 애플은 Xcode 의 "Convert to App Intent..." 변환기와 [CustomIntentMigratedAppIntent](#) 프로토콜(기존 단축어·도네이션을 계속 살려주는 호환 장치)을 제공한다.

"Starting with iOS 17 and macOS 14, you use App Intents to create configurable widgets. [...] If the minimum version your app supports is iOS 17, macOS 14, or watchOS 10, use App Intents for your widget configurations."

SiriKit

UMCApp — 이 앱은 이걸로 무엇을 하는가

UMCApp 에는 SiriKit 도, 그 후속인 App Intents 도 붙어 있지 않다. `import SiriKit / import Intents / import AppIntents` 는 한 줄도 없고, WidgetKit 을 import 하는 6개 파일(`UMCAppWidget/Sources/` 3개, `Core/WidgetShared/Sources/Entries/` 3개)이 전부다. 홈 화면 위젯인 `UMCHomeWidget` 은 `UMCAppWidget/Sources/UMCAppWidget.swift:15` 에서 `StaticConfiguration` 을 쓰는데, 이건 "사용자가 위젯을 길게 눌러 설정할 항목이 없는" 고정형이라는 뜻이다 — 설정 가능 위젯이었다면 `AppIntentConfiguration` + `WidgetConfigurationIntent` 가 등장했을 자리이고, 그게 이 앱에서 App Intents 를 만날 수 있었던 유일한 지점이었다.

`UMCApp.entitlements` 에는 `aps-environment` ·Apple 로그인·iCloud/CloudKit·`com.apple.developer.nearby-interaction` ·App Group 만 있고 `com.apple.developer.siri` 가 없으며, `Project.swift` 의 `infoPlist` 에도 `NSUserActivityTypes` 키가 없다 — 레거시 SiriKit(Intents 익스텐션) 경로가 붙을 자리가 양쪽 다 비어 있다는 뜻이다. 후속인 App Intents 는 이 두 키를 요구하지 않지만, `import AppIntents` 자체가 0건이라 그쪽 경로도 없다.

`.intentdefinition` 파일도, Intents App Extension 타겟도 없다(`Tuist/ProjectDescriptionHelpers/Project+WidgetExtension.swift:38-40` 이 만드는 유일한 익스텐션의 `NSExtensionPointIdentifier` 는 `com.apple.widgetkit-extension` 이다). 그래서 사용자가 "시리아, UMC 출석해 줘" 라고 말했을 때 이 앱이 반응하는 경로는 현재 존재하지 않고, 앱 밖에서 앱 안으로 들어오는 길은 오직 `umc://` 커스텀 URL 스킴(`UMCApp/Sources/UMCAppApp.swift:88` 의 `.onOpenURL` → `AppDeepLink.parse` → `DeepLinkStore`)과 FCM 푸시뿐이다. 참고로 동결된 레거시 `AppProduct` 에도 Siri 흔적은 없어서, "예전에 있었다가 뺐다" 가 아니라 애초에 도입한 적이 없는 기술이다.

UMCAppWidget/Sources/UMCAppWidget.swift

L8-22

```
import WidgetKit
import SwiftUI

struct UMCAppWidget: Widget {
    let kind: String = "UMCHomeWidget"

    var body: some WidgetConfiguration {
        StaticConfiguration(kind: kind, provider: UMCAppWidgetProvider()) { entry in
            UMCAppWidgetEntryView(entry: entry)
        }
        .configurationDisplayName("UMC")
        .description("UMC 동아리 정보를 홈 화면에서 확인하세요.")
        .supportedFamilies([.systemSmall, .systemMedium])
    }
}
```

부재의 결정적 증거. 위젯이 `StaticConfiguration` 을 쓴다 — 사용자가 위젯을 설정할 수 있게 하려면 여기가 `AppIntentConfiguration(kind:intent:provider:)` 이어야 하고, 그 `intent:` 자리에 `WidgetConfigurationIntent` 를 채택한 App Intents 타입이 들어간다. 이 앱에서 App Intents 를 만날 수 있었던 가장 자연스러운 자리가 비어 있다는 뜻이다.

UMCApp/Sources/Navigation/DeepLinkStore.swift

L23-43

```
var pending: AppDeepLink?

// MARK: - Function

/// 링크를 세워 둔다. 내부 링크가 아니면 아무 일도 하지 않는다.
///
/// - Returns: 내부 링크로 인식해 받아 뒀는지 여부, `false` 면 다른 핸들러(소셜 로그인)가
```

```

/// 처리할 URL 이다.
@discardableResult
func receive(_ url: URL) -> Bool {
    guard let link = AppDeepLink.parse(url) else { return false }
    pending = link
    return true
}

/// 세위 둔 링크를 꺼내 비운다.
func take() -> AppDeepLink? {
    defer { pending = nil }
    return pending
}
}

```

Siri 대신 이 앱이 실제로 쓰는 '바깥에서 앱 안으로' 진입로. `umc://card/{memberId}` 같은 URL 을 받아 탭 셀이 뜰 때까지 한 칸짜리 우편함에 세위 뒀다가 꺼낸다. 나중에 App Intents 를 붙인다면 URL 문자열을 새로 파싱할 필요 없이 `AppDeepLink` 를 여기에 그대로 밀어 넣는 방식으로 재사용할 수 있는 구조다(아래 designNotes 의 제안 B).

설계에서의 선택

- 프로젝트가 App Intents 를 '몰라서' 안 쓴 게 아니라 의식적으로 우선순위에서 뺐다는 근거가 있다. 모노레포 루트의 `../docs/claude/ios26-
frameworks/INDEX.md` 는 iOS 26 프레임워크 문서 21개를 표로 정리하면서, 30번째 줄에서 `AppIntents-Updates.md` 를 '그 외 프레임워크 문서 (필요 시 참고)' 구획에 넣었다. 반대로 같은 문서가 '이 프로젝트에 특히 관련 높은 문서' 로 승격시킨 건 Liquid Glass, WidgetKit, Swift Concurrency, FoundationModels 쪽이다. 즉 이 앱의 기술 예산은 화면·디자인 시스템·동시성에 쏠려 있고 시스템 통합(Siri/단축어)은 뒤로 미룬 상태다.
- 한 가지 유리한 조건은 있다. `Project.swift` 의 `deploymentTargets:` `.iOS("26.4")` 라서, 애플 문서가 경고하는 '구버전 OS 하위 호환' 문제 (`IntentConfiguration` 과 `AppIntentConfiguration` 을 `#available` 로 분기해 두 별 유지)가 이 앱에는 아예 발생하지 않는다. 도입한다면 레거시 SiriKit `INIntent` 는 건드릴 필요 없이 App Intents 만 새로 쓰면 된다. 반대로 말하면 지금 안 하고 있는 건 순수한 우선순위 문제지 기술적 제약이 아니다.
- [제안 A — 가장 자연스러운 후보: 출석]**
`Features/Activity/Domain/Sources/UseCases/ChallengerAttendanceUseCaseProtocol.swift` 의 `fetchAvailableSchedules(now:)` 로 '지금 출석 가능한 세션'을 뽑고 `requestGPSAttendance(...)` 로 찍는 흐름은, 부원이 세션장 문 앞에서 앱을 열고 탭 3번 하는 대신 잠금화면 단축어 한 번으로 끝낼 수 있는 전형적인 App Intents 소재다. 다만 트레이드오프가 크다 — 이 UseCase 는 `LocationProviding` 지오펜스 판정에 의존하고(주석: "특정 일정 출석 가능 여부는 `requestGPSAttendance` 내부의 `isInside(geofenceId:)` 식별자 기반 판정을 사용합니다."), 위치 권한이 없으면 실패한다. Siri 가 백그라운드에서 조용히 출석을 대신 찍어주는 형태는 위치 권한·오탐(실제로는 안 갔는데 찍히는 것) 리스크가 있으므로, `OpenIntent` 로 출석 화면까지만 열어 주고 최종 탭은 사람이 하게 두는 쪽이 현실적이다.
- [제안 B — 가장 저렴한 후보: 딥링크 재사용]**
`UMCApp/Sources/Navigation/AppDeepLink.swift` 와 `DeepLinkStore.swift` 는 이미 '바깥에서 들어온 목적지 하나를 보관했다가 탭 셀이 뜨면 연다'는 구조를 갖고 있다. 여기에 `AppIntent` 하나를 얹어 `perform()` 이 URL 파싱을 건너뛰고 `DeepLinkStore.pending` 에 `AppDeepLink.card(memberId:)` 나 `.message(...)` 를 직접 세위 두게 하면, '내 명함 보여줘' / '스레드 열어줘' 같은 동작이 코드 중복 없이 단축어·Spotlight 에 노출된다. 화면 전환 로직을 새로 짤 필요가 없다는 게 이 경로의 장점이다. 단, 이걸 어디까지나 **제안**이며 현재 코드에는 이런 의도가 적힌 주석도, 계획 문서도 없다.

함께 맞물리는 기술 `WidgetKit`

쓰지 않은 것 · 한계

- SiriKit 을 안 쓰는 것과 별개로, 이 앱은 App Intents 가 열어 주는 시스템 표면 전부를 포기하고 있다 — Spotlight 검색 결과에 앱 동작이 뜨지 않고, 단축어 앱에 UMCApp 액션이 하나도 없으며, iPhone 15 Pro 이상의 액션 버튼이나 컨트롤 센터 컨트롤(`ControlWidget`)에도 등록할 수 없다. `AppEntity` 가 없으니 '내가 받은 명함' 같은 앱 내부 데이터를 시스템이 검색·참조할 수도 없다.
- 현재 홈 화면 위젯 자체가 미완성 스텝이다.
`UMCAppWidget/Sources/UMCAppWidgetEntryView.swift:22` 가 '위젯 준비 중입니다' 를 렌더링하고, `UMCAppWidgetProvider.swift:25` 의 타임라인 정책은 `.never` 다. `Core/WidgetShared/Sources/Entries/` 에 `WidgetScheduleEntry` · `WidgetNoticeEntry` · `WidgetAttendanceEntry` 세 개의 엔트리 타입이 미리 준비돼 있지만 어느 것도 실제 위젯에 연결돼 있지 않다. 위젯 본체가 이 상태이므로 '설정 가능 위젯 → `AppIntentConfiguration`' 논의는 아직 시기상조다.
- Apple Watch 쪽도 마찬가지로 비어 있다. `UMCWatchApp/Sources/` 에는 `UMCWatchApp.swift` 와 `ContentView.swift` 두 파일뿐이라, SiriKit 이 watchOS 에서 제공하던 Siri 위치 페이스 카드나 relevant shortcuts 같은 기능은 불일 대상 자체가 없다.

SiriKit <https://developer.apple.com/documentation/sirikit>
App Intents <https://developer.apple.com/documentation/appintents>

App intents (API Collection) <https://developer.apple.com/documentation/appintents/app-intents>

Migrating widgets from SiriKit Intents to App Intents <https://developer.apple.com/documentation/widgetkit/migrating-from-sirikit-intents-to-app-intents>



Nearby Interaction

부분 적용

초광대역 정밀 측위

명함 교환 목록에서 상대까지의 거리(예: 2.1m)를 UWB로 실측해 보여준다.

Apple 공식 문서 — 이 기술은 무엇인가

Nearby Interaction(NI)은 UWB(초광대역) 칩이 있는 애플 기기끼리 서로의 상대 위치 — 거리(distance)와 방향(direction) — 을 주기적으로 측정하게 해 주는 프레임워크다. 핵심 타입은 `NISession` 이며, Apple 문서는 이를 "An object that identifies a unique connection between two peer devices" 로 정의한다. **NI 자체는 상대를 찾아주지도, 데이터를 날라주지도 않는다.** 앱이 직접 다른 네트워크 기술(Multipeer Connectivity, Core Bluetooth, Watch Connectivity, 자체 서버 등)로 상대를 발견하고 세션의 `discoveryToken` 을 주고받아야 비로소 `GINearbyPeerConfiguration(peerToken:)` 으로 세션을 실행할 수 있다. 값은 `NISessionDelegate` 콜백(`session(_:didUpdate:)`)으로 흘러오고, 실행 전에 `NSNearbyInteractionUsageDescription` 기반 권한 동의가 필요하다.

"To perform this discovery and handshake, the app implements custom code using a network technology you choose. Common examples are Core Bluetooth, Multipeer Connectivity, Watch Connectivity, or a custom server deployment."

Nearby Interaction

UMCApp — 이 앱은 이걸로 무엇을 하는가

UMC 부원이 마이페이지 탭에서 내 명함의 「명함 교환」 버튼을 누르면 (`MyPageView.swift:128`) `BusinessCardDestination.exchange` 로 이동해 `CardExchangeView` 가 열리고, `.task` 가 `CardExchangeViewModel.start()` 를 돌린다. 이 시점에 `ExchangeCardsUseCase` 가 `MPCTransport` 로 Bonjour 서비스 `_umc-card._tcp` 광고-탐색을 시작하고, 동시에 `PeerRangingCoordinator.start(preview:)` 로 내 미리보기를 세팅한 뒤 거리 스트림을 미리 구독한다.

MPC 가 주변 기기를 발견하면 즉시 연결을 걸고, 연결이 서는 순간 `MPCTransport.session(_:peer:didChange:)` 가 `makeHandshake(forPeerID:)` 를 불러 그 피어 전용 `NISession` 을 새로 만들고 `discoveryToken` 을 직렬화해 `NearbyMessage.handshake` 로 보낸다 — 즉 UWB 토큰의 운반 채널은 MultipeerConnectivity 다. 상대 핸드셰이크가 도착하면 `didReceiveHandshake(_:fromPeerID:)` 가 토큰을 역직렬화해 `session.run(GINearbyPeerConfiguration(peerToken:))` 을 호출하고, 그때부터 `session(_:didUpdate:)` 가 초당 여러 번 `object.distance` 를 던진다.

이 값은 `PeerDistance` → `ExchangeEvent.distanceUpdated` → `CardExchangeViewModel.updateDistance` 를 타고 `DiscoveredPeerRow` 오른쪽의 「2.1m」 텍스트로 나온다(시안 Figma 12654:32621). 사용자가 그 행을 탭해야 비로소 명함 본체(`ExchangePayload`)가 MPC 로 건너가고, 상대 명함이 도착하면 `ExchangeCompletedView` 가 전체화면으로 덮는다 — 거리는 교환과 완전히 분리된 별개 흐름이다.

부원이 보는 화면

마이페이지 → 내 명함의 「명함 교환」 → 「명함 교환」 화면. 주변 부원이 목록에 한 줄씩 뜨고, 각 행 오른쪽에 신호 막대와 함께 「2.1m」 같은 실측 거리가 붙는다. 그 행을 누르면 그 사람에게 내 명함이 건너간다. UWB 가 없는 기기(구형 아이폰·아이패드)나 범위 밖이면 막대만 남고 숫자 칸이 비어 있다.

코드에서의 위치

파일	심볼	역할
Core/NearbyExchange/Sources/Ranging/ PeerRangingCoordinator.swift:48	<code>PeerRangingCoordinator</code>	앱 안에서 <code>NISession</code> 을 다루는 유일한 제품 코드. 피어마다 세션 하나를 만들어 거리를 <code>AsyncStream<PeerDistance></code> 로 흘린다.
Core/NearbyExchange/Sources/Ranging/ PeerRangingCoordinator.swift:149	<code>PeerRangingCoordinator.makeHandshake(forPeerID:)</code>	피어 전용 <code>NISession</code> 을 만들고 그 <code>discoveryToken</code> 을 <code>NSKeyedArchiver</code> 로 직렬화해 핸드셰이크에 실는다.
Core/NearbyExchange/Sources/Ranging/ PeerRangingCoordinator.swift:221	<code>PeerRangingCoordinator.session(_:didUpdate:)</code>	<code>NISessionDelegate</code> 콜백. <code>object.distance</code> 만 읽어 <code>PeerDistance</code> 로 방출한다 (direction 은 안 쓴다).
Core/NearbyExchange/Sources/Transports/ MPCTransport.swift:698	<code>MPCTransport.session(_:peer:didChange:)</code>	MPC 연결이 서면 <code>makeHandshake</code> 로 만든 NI 토큰을 상대에게 전송한다 — 토큰이 건너가는 실제 채널.

파일	심볼	역할
Core/NearbyExchange/Sources/Models/NearbyMessage.swift:68	NearbyHandshake.niToken	MPC 위로 오가는 봉투에 실리는 직렬화된 NIDiscoveryToken(실측 343B). UWB 미탐재 기기는 nil.
Features/BusinessCard/Domain/Sources/UseCases/Implementations/ExchangeCardsUseCase.swift:55	ExchangeCardsUseCase.init / start(myCard:)	MPCTransport 에 ranging 을 handshakeProvider 로 쫓고 (55행), 거리 스트림을 .distanceUpdated 이벤트로 변환한다(90-96행, 방출은 95행).
Features/BusinessCard/Presentation/Sources/ViewModels/CardExchangeViewModel.swift:142	CardExchangeViewModel.updateDistance(peerID:meters:)	.distanceUpdated 를 받아 해당 피어 행의 distanceMeters 만 제자리 갱신한다.
Features/BusinessCard/Presentation/Sources/Components/DiscoveredPeerRow.swift:102	DiscoveredPeerRow.signal	화면 끝단. 거리가 있으면 '2.1m', nil 이면 숫자 없이 신호 막대만 그린다.

Core/NearbyExchange/Sources/Ranging/PeerRangingCoordinator.swift	L183-201
<pre> /// 상태 토큰으로 레인징을 시작한다. public func didReceiveHandshake(_ handshake: NearbyHandshake, fromPeerID peerID: String) { #if canImport(NearbyInteraction) guard let data = handshake.niToken else { return } // 상대가 UWB 미탐재 guard let session = stateQueue.sync(execute: { sessionsByPeerID[peerID] }) else { // 내 토큰을 아직 안 만든 피어다. 순서가 어긋난 것이라 무시한다 - // 연결 시 makeHandshake 가 먼저 오는 게 정상 흐름이다. record("세션 없는 피어의 핸드셰이크: \(peerID)") return } guard let token = try? NSKeyedUnarchiver.unarchivedObject(ofClass: NIDiscoveryToken.self, from: data) else { record("NI 토큰 역직렬화 실패: \(peerID)") return } session.run(NINearbyPeerConfiguration(peerToken: token)) #endif } </pre>	

MPC 로 받은 상대의 discoveryToken 을 역직렬화해 그 피어 전용 NISession 을 실행하는 지점. 상대가 UWB 미탐재라 토큰이 nil 이면 조용히 넘어가 거리 없이 목록만 동작한다.

Core/NearbyExchange/Sources/Transports/MPCTransport.swift	L686-700
<pre> guard state == .connected else { return } // 연결되면 핸드셰이크를 보낸다. 여기 실리는 건 NI 토큰과 미리보기용 - 명함이 아니다. // 토큰은 피어마다 다르므로 그때그때 만든다 (NearbyHandshakeProviding 주석 참고). // 초대를 수락해 연결된 피어는 발견 경로를 안 거쳤을 수 있다. 여기서 매핑을 보강해 // 두면 이후 `send(payload:to:)` 가 그 피어를 찾지 못하는 일이 없다. let sessionID = peerID.displayName let provider = stateQueue.sync { () -> (any NearbyHandshakeProviding)? in peersBySessionID[sessionID] = peerID return handshakeProvider } guard let provider, let handshake = provider.makeHandshake(forPeerID: sessionID) else { return } try? sendMessage(.handshake(handshake), to: [peerID], session: session) } </pre>	

UWB 토큰이 실제로 건너가는 채널. MultipeerConnectivity 세션이 연결되는 순간 피어별 NI 토큰을 만들어 보낸다 — Apple 문서가 말하는 "discovery and handshake" 를 MPC 로 구현한 부분이다.

설계에서의 선택

- NI 를 transport 로 만들지 않았다. PeerRangingCoordinator.swift:33-35 주석: "NearbyInteraction 은 데이터를 나르지 못한다. 거리-방향만 준다. 그래서 NearbyTransportProtocol 구현체가 될 수 없고(그렇게 두면 영원히 구현 불가), transport 옆에 붙어 거리만 보태는 별도 계층이 맞다." 그래서 DI 도 transport 와 별개로 PeerRangingCoordinator 를 등록한다 (UMCApp/Sources/DIContainer+BusinessCard.swift:58-62).

쓰지 않은 것 · 한계

- 방향(direction)은 제품에서 안 쓴다. 제품 경로인 PeerRangingCoordinator 는 object.distance 만 읽는다(:226). object.direction 을 읽는 코드는 UMCApp/Sources/Debug/BusinessCard/NearbyRangingController.swift:134 뿐이고 그 파일 전체가 #if DEBUG 로 묶여 있다. Camera Assistance(isCameraAssistanceEnabled · setARSession)와 supportsDirectionMeasurement 는 레포 어디에도 없다 — 즉 「상대 쪽을 가리키는 화살표」 같은 경험은 만들지 않았다.

- 세션은 피어마다 하나. `INearbyPeerConfiguration(peerToken:)` 이 상대 토큰 하나만 받으므로 상대가 늘면 세션도 늘고 세션마다 `discoveryToken` 이 다르다. 전역 토큰 하나를 뿌리면 두 번째 상대부터 영동한 세션 토큰을 받는다 (`MPCTransport.swift:15-18`). `sessionsByPeerID` / `peerIDsBySession` 양방향 딕셔너리가 이 때문에 존재한다.
- NI 는 대칭이라 토큰 교환을 자동화했다. `PeerRangingCoordinator.swift:43-47` : "양쪽이 서로의 토큰으로 세션을 실행해야 값이 나온다. 한쪽만 실행하면 양쪽 다 아무것도 받지 못한다. 2026-08-16 실기기 검증에서 이 조건을 몰라 한참 헤맸다 — 그래서 토큰 교환을 자동화한다. 사용자가 순서를 틀릴 여지를 없애는 것이 이 계층의 존재 이유다." 그 결과 MPC 는 발견 즉시 연결부터 건다(`MPCTransport.swift:608-609`).
- 동의 수준을 3단으로 나눴다. 발견(discoveryInfo) → 연결(핸드셰이크: 미리보기 + NI 토큰, 자동) → 전송(명함 전체, 행을 탭해야). NI 토큰(343B)은 Bonjour TXT 레코드에 안 들어가서 세션이 열린 뒤에만 보낼 수 있고, 토큰이 없으면 거리를 못 잰다 — 연결을 미리 해 두는 이유가 이것이다(`MPCTransport.swift:41-48`). 그래서 자동으로 오가는 `PeerPreview` 에는 이메일·외부 링크를 일부러 넣지 않았다(`MyCard+PeerPreview.swift:15-16`).

- UWB 미지원 기기용 안내가 없다. `PeerRangingCoordinator.isSupported` (:53)는 public 이지만 실제로 읽는 곳은 같은 파일 :153 하나뿐이고, 어떤 View 도 이 값을 보지 않는다. 그래서 아이패드나 구형 아이폰에서는 '이 기기는 거리 측정을 못 한다'는 설명 없이 그냥 숫자 칸이 조용히 비어 있다. QR(`CardQRView` / `CardLinkReceiver`)은 UWB 폴백이 아니라 딥링크로 명함을 주는 별개 경로다 — DEBUG 화면 `NearbyRangingSection` 만이 QR 로 NI 토큰을 교환하는데, 이는 페어링·capability 없이 UWB 자체를 검증하려는 계층용이다 (`NearbyRangingSection.swift:15-17`).
- NI 경로에 대한 자동 테스트가 없다. `Core/NearbyExchange/Tests/` 5개 파일 어디에도 `PeerRangingCoordinator` 테스트가 없고, 검증되는 건 `NearbyHandshake` 의 Codable 왕복(토큰이 있을 때/nil 일 때)뿐이다 (`NearbyMessageTests.swift:52,70`). 상위도 마찬가지로 `ExchangeCardsUseCase(ranging:)` 기본값이 `nil` 이라 테스트는 NI 없이 돈다. `NISession` 이 시뮬레이터에서 동작하지 않는 데서 오는 한계이고, 대신 `PeerRangingCoordinator.lastError` (:78)도 어디서도 읽히지 않아 실패 원인은 코드로는 노출되지 않는다.

Nearby Interaction <https://developer.apple.com/documentation/nearbyinteraction>

Initiating and maintaining a session <https://developer.apple.com/documentation/nearbyinteraction/initiating-and-maintaining-a-session>

NI Session <https://developer.apple.com/documentation/nearbyinteraction/nisession>

NI DiscoveryToken <https://developer.apple.com/documentation/nearbyinteraction/nidiscoverytoken>



CARD 09 / 09

WidgetKit

부분 적용

위젯·라이브 액티비티

홈 화면 위젯 확장 타겟은 실제로 빌드·탑재되지만, 아직 "준비 중" 안내만 띄운다

Apple 공식 문서 — 이 기술은 무엇인가

WidgetKit은 앱의 정보를 홈 화면·잠금 화면·Apple Watch 문자판 등 앱 바깥의 시스템 영역에 작은 카드로 내보내는 프레임워크다. 위젯은 항상 실행되는 게 아니라, 앱이 미리 만들어 둔 "타임라인(Timeline)" — 즉 `TimelineEntry` 들의 배열 — 을 시스템이 받아 두었다가 정해진 시각에 화면을 갈아끼우는 방식으로 동작한다. 화면은 SwiftUI 로 그리지만 상태를 갖지 않으며(stateless), 사용자 조작에 즉시 반응하는 대신 갱신 정책(`.atEnd` / `.after(·)` / `.never`)으로 다음 갱신 시점을 시스템에 알려준다. 위젯은 앱 본체와 별개의 프로세스(App Extension)라서, 데이터를 넘기려면 App Group 같은 공유 저장소가 필요하다.

"Because the character's health has reached 100 percent, the provider responds with a single entry for the current time and a refresh policy set to never. With this setting, WidgetKit doesn't ask for another timeline until the app uses WidgetCenter to tell WidgetKit to request a new timeline."

WidgetKit

UMCApp — 이 앱은 이걸로 무엇을 하는가

UMC 부원이 아이폰 홈 화면을 길게 눌러 위젯 갤러리에서 "UMC"를 고르면, 시스템은 앱 번들 안에 임베드된 `UMCAppWidget` 확장을 깨우고 `@main UMCAppWidgetBundle` → `UMCHomeWidget` 의 `StaticConfiguration` 을 읽는다. 거기 연결된 `UMCHomeWidgetProvider` 가 `placeholder(in:) / getSnapshot(in:completion:)` / `getTimeline(in:completion:)` 세 콜백에 모두 `UMCHomeWidgetEntry(date: .now)` 하나만 담아 돌려주고, 타임라인 갱신 정책은 `.never` 다.

그 엔트리를 받아 `UMCHomeWidgetEntryView` 가 그리는 화면은 "UMC" 제목과 "위젯 준비 중입니다." 한 줄이 전부다 — 즉 홈 화면에 실제로 카드가 뜨긴 하지만 앱 데이터는 단 한 글자도 실려 있지 않다. 데이터를 실어 나를 배관은 `Core/WidgetShared` 에 미리 깔려 있다: `WidgetStorage` 가 App Group `group.com.umc.product.widget` 의 `UserDefaults(suiteName:)` 을 감싸고, 공지·일정·출석 세 가지 `TimelineEntry` 모델(`WidgetNoticeEntry` , `WidgetScheduleEntry` , `WidgetAttendanceEntry`)이 정의돼 있으며, App Group entitlement 도 `UMCApp.entitlements` 와 `UMCAppWidget.entitlements` 양쪽에 들어가 있다.

그러나 `grep -rn "WidgetStorage"` / `grep -rn "import CoreWidgetShared"` 결과가 0건이라, 홈 탭(`HomeView`)이나 공지 탭(`NoticeView`)의 ViewModel 이 이 저장소에 값을 써 주는 경로는 아직 존재하지 않는다. 정책이 `.never` 인데 `WidgetCenter.shared.reloadTimelines(ofKind:)` 호출부도 레포 전체에 0건이므로, 지금 구조에서는 앱에서 무슨 일이 일어나도 위젯이 다시 그려질 계기 자체가 없다.

부원이 보는 화면

홈 화면을 길게 눌러 위젯 갤러리에 들어가면 "UMC" 항목이 보이고 설명에 "UMC 동아리 정보를 홈 화면에서 확인하세요."가 뜬다. 소형(systemSmall)·중형(systemMedium) 두 크기를 고를 수 있지만, 실제로 추가하면 카드 안에는 "UMC / 위젯 준비 중입니다." 문구만 나온다. 공지·일정·출석 정보는 아직 표시되지 않는다.

코드에서의 위치

파일	심볼	역할
UMCAppWidget/Sources/ UMCAppWidgetBundle.swift:12	<code>UMCAppWidgetBundle</code>	@main WidgetBundle — 위젯 확장의 진입점. 현재 위젯 1개(UMCHomeWidget)만 등록
UMCAppWidget/Sources/ UMCAppWidget.swift:11	<code>UMCHomeWidget</code>	StaticConfiguration 로 위젯 정의. 갤러리 표시명 "UMC", systemSmall/systemMedium 만 지원
UMCAppWidget/Sources/ UMCAppWidgetProvider.swift:14	<code>UMCHomeWidgetProvider</code>	TimelineProvider 구현. 엔트리 1개짜리 Timeline 을 policy: .never 로 반환 (자동 갱신 없음)
UMCAppWidget/Sources/ UMCAppWidgetEntryView.swift:10	<code>UMCHomeWidgetEntryView</code>	위젯 본문 SwiftUI 뷰. "UMC / 위젯 준비 중입니다." 고정 텍스트만 그린다
Core/WidgetShared/Sources/ Storage/WidgetStorage.swift:14	<code>WidgetStorage</code>	App Group(group.com.umc.product.widget) UserDefaults 래퍼. 앱↔위젯 데이터 교환용으로 만들어졌으나 호출부 0곳
Core/WidgetShared/Sources/ Entries/ WidgetNoticeEntry.swift:11	<code>WidgetNoticeEntry</code>	공지 위젯용 TimelineEntry 모델. 같은 폴더에 WidgetScheduleEntry(:11), WidgetAttendanceEntry(:11) 존재. 셋 다 아직 어디서도 참조되지 않음
Tuist/ProjectDescriptionHelpers/ Project+WidgetExtension.swift:16	<code>widgetExtensionProject(name:bundleId:entitlements:dependencies:)</code>	appExtension 타겟 + NSExtensionPointIdentifier=com.apple.widgetkit-extension Info.plist 를 자동 생성하는 Tuist 헬퍼

파일	심볼	역할
Project.swift:131	UMCApp target dependencies	앱 타겟이 UMCAppWidget 프로젝트를 의존성으로 선언 → 확장이 앱 번들에 임베드된다

UMCAppWidget/Sources/UMCAppWidgetProvider.swift	L10-28
<pre> struct UMCHomeWidgetEntry: TimelineEntry { let date: Date } struct UMCHomeWidgetProvider: TimelineProvider { func placeholder(in context: Context) -> UMCHomeWidgetEntry { UMCHomeWidgetEntry(date: .now) } func getSnapshot(in context: Context, completion: @escaping (UMCHomeWidgetEntry) -> Void) { completion(UMCHomeWidgetEntry(date: .now)) } func getTimeline(in context: Context, completion: @escaping (Timeline<UMCHomeWidgetEntry>) -> Void) { let entry = UMCHomeWidgetEntry(date: .now) let timeline = Timeline(entries: [entry], policy: .never) completion(timeline) } } </pre>	

위젯의 심장인 TimelineProvider 전문. 엔트리에 date 말고는 아무 데이터도 없고, 세 콜백 모두 현재 시각 엔트리 하나만 돌려준 뒤 policy: .never 로 "다시 물어보지 마라"고 시스템에 알린다. WidgetKit 배선은 정상이지만 실을 내용물이 아직 비어 있는 상태다.

Core/WidgetShared/Sources/Storage/WidgetStorage.swift	L10-28
<pre> /// App Group 기반 위젯 공유 스토리지 래퍼 /// /// `group.com.umc.product.widget` App Group을 통해 /// 메인 앱과 Widget Extension 간 데이터를 공유합니다. public final class WidgetStorage: Sendable { // MARK: - Property public static let shared = WidgetStorage() private static let appGroupSuiteName = "group.com.umc.product.widget" nonisolated(unsafe) private let defaults: UserDefaults? // MARK: - Init public init() { self.defaults = UserDefaults(suiteName: Self.appGroupSuiteName) } </pre>	

앱과 위젯이 데이터를 주고받을 통로로 준비된 WidgetStorage. SwiftData 공유 컨테이너가 아니라 App Group UserDefaults + JSONEncoder/JSONDecoder 조합이며(이어지는 save/load 가 제네릭 Encodable/Decodable 을 받는다), 실제 호출부는 레포 전체에 아직 한 곳도 없다.

설계에서의 선택

- 데이터 공유 방식으로 SwiftData 공유 컨테이너가 아니라 App Group UserDefaults + JSON 인코딩을 골랐다 ([WidgetStorage.swift:20,27](#)). 위젯이 읽어야 할 값이 '공지 1건·일정 1건·출석 상태 1건' 처럼 아주 작고 읽기 전용이라 스키마-마이그레이션 부담이 있는 SwiftData 보다 가법다는 트레이드오프다. 대신 쿼리-관계는 못 쓰고, 저장 키는 [WidgetStorageKey](#) enum([widget.scheduleEntry](#) 등, :53-57)으로 문자열 오타를 막는 정도만 보장한다.
- Xcode 14.1 주석에 App Extension 배포 합정이 명시돼 있다 — [Project+WidgetExtension.swift:35-37](#) : "앱 익스텐션의 CFBundleVersion은 호스트 앱과 반드시 일치해야 한다. 어긋나면 App Store Connect 업로드가 거부된다." 그래서 위젯 Info.plist 의 버전을 [\\$\(CURRENT_PROJECT_VERSION\)](#) 로 앱과 같은 변수에 묶었고, [NSExtensionPointIdentifier](#) (com.apple.widgetkit-extension)도 헬퍼가 자동으로 박아 개별 타겟이 실수할 여지를 줄였다.

쓰지 않은 것 · 한계

- 실사용 데이터가 위젯에 전혀 흐르지 않는다. `grep -rn "WidgetStorage"` 는 정의 파일 자신(Core/WidgetShared/Sources/Storage/WidgetStorage.swift) 외에 0건이고, `grep -rn "import CoreWidgetShared"` 도 0건이다. 즉 [WidgetNoticeEntry](#) / [WidgetScheduleEntry](#) / [WidgetAttendanceEntry](#) 세 모델은 정의만 되어 있고 생성·저장·소비되는 지점이 하나도 없다. 홈·공지 화면의 ViewModel → App Group → 위젯으로 이어지는 사슬은 아직 끊겨 있다.
- [WidgetCenter](#) 가 레포 전체에서 0건이다(`grep -rn --include="*.swift" "WidgetCenter"`). 타임라인 정책이 `.never` 이므로, 나중에 데이터를 채우더라도 [WidgetCenter.shared.reloadTimelines\(ofKind: "UMCHomeWidget"\)](#) 를 앱 쪽(예: 공지 수신 AppDelegate, 출석 체크 완료 시점)에 심지어 않으면 위젯은 갱신되지 않는다.

- 위젯 확장자의 번들 ID 를 `com.umc.product.widget` 으로 잡아 호스트 앱 `com.umc.product` 를 prefix 로 갖게 했고 (`UMCAppWidget/Project.swift:6` , 헬퍼 주석 :13), App Group 도 같은 체계인 `group.com.umc.product.widget` 로 통일했다. 최초 커밋에서는 `group.dev.umc.appproduct.widget` 이었는데 커밋 b291540e "App Group 식별자를 번들 ID 체계에 맞게 통일"로 갈아탄 이력이 있다.
- 갱신 정책을 `.never` 로 둔 것은 현재 상태에서는 일관된 선택이다. 표시할 데이터가 없으니 시스템이 주기적으로 확장을 깨울 이유가 없고, Apple 문서도 `.never` 는 앱 쪽에서 `WidgetCenter.reloadTimelines(ofKind:)` 로 밀어 주는 방식과 짝을 이루라고 안내한다. 다만 그 짝인 `reloadTimelines` 호출부가 아직 없어서, 데이터를 채우는 순간 함께 넣지 않으면 위젯이 영원히 안 바뀌는 형태로 남는다.

- 지원 family 가 `.systemSmall` / `.systemMedium` 두 개뿐이다 (`UMCAppWidget.swift:20`). `.systemLarge` 와 잠금 화면·Apple Watch 용 `.accessoryCircular` / `.accessoryRectangular` / `.accessoryInline` 은 지원하지 않는다. 또 `AppIntentConfiguration` (사용자가 위젯을 편집해 표시 항목을 고르는 기능)이 아니라 `StaticConfiguration` 이라 위젯 설정 커스터마이징이 불가능하고, `widgetURL` / `Link` 딥링크나 `Button` / `Toggle` 인터랙티브 위젯도 없어 위젯을 눌러도 앱 첫 화면만 열린다. iOS 26 Liquid Glass 대응(`widgetRenderingMode` , `widgetAccentable` , `containerBackground`)도 코드에 반영돼 있지 않다 — 레포에 관련 가이드 문서 (`../docs/claude/ios26-frameworks/WidgetKit-Implementing-Liquid-Glass-Design.md`)만 있고 적용은 안 됐다.
- Live Activity(ActivityKit)는 실제로 0건이다 — `grep -rn -e "ActivityKit" -e "ActivityAttributes"` 결과 없음. 잠금 화면/Dynamic Island 에 GPS 출석 진행 상황을 띄우는 식의 기능은 구현돼 있지 않다. Apple Watch 쪽(`UMCWatchApp/Sources/` , 파일 2개)에도 `WidgetKit` import 가 없어 워치 컴플리케이션도 없으며, 워치는 `CoreWatchConnectivity` 만 의존한다.

WidgetKit <https://developer.apple.com/documentation/widgetkit>
TimelineProvider <https://developer.apple.com/documentation/widgetkit/timelineprovider>

WidgetCenter <https://developer.apple.com/documentation/widgetkit/widgetcenter>
Creating a widget extension <https://developer.apple.com/documentation/widgetkit/creating-a-widget-extension>

9가지 판정과 각 기술의 대표 코드 위치.

기술	판정	대표 위치	이 앱에서 하는 일
01 CoreLocation 위치·방향	핵심	Core/Foundation/Sources/Location/LocationManager.swift	세션 장소 반경 50m 안에 있어야만 출석 버튼이 통과되는 GPS 스마트 출석의 심장
02 Apple Intelligence 온디바이스 개인 지능	부분 적용	Features/Community/Data/Sources/AI/FoundationModelsThreadClassifier.swift	기기가 Apple Intelligence를 쓸 수 있을 때만 AI 기능을 열어 주는 관문 역할
03 CloudKit iCloud 동기화	부분 적용	UMCApp/Sources/UMCAppApp.swift	명함첩·공지 읽음 표시 같은 기기 로컬 기록을 iCloud로 자동 동기화한다.
04 CoreML 온디바이스 머신러닝	부분 적용	Features/Home/Data/Sources/Repositories/ScheduleClassifierRepository.swift	일정 제목·푸시 문구를 기기 안에서 분류해 아이콘과 태그를 정한다
05 Foundation Models 내장 LLM (iOS 26)	핵심	Features/Community/Data/Sources/AI/FoundationModelsThreadSummarizer.swift	기기 안에서 도는 AI가 일정·공지·스레드를 대신 쓰고 요약하고 분류한다.
06 Liquid Glass iOS 26 디자인	핵심	UMCApp/Sources/RootTab/RootTabView.swift	앱의 거의 모든 카드·버튼·툴바 표면을 그리는 기본 재질이다.
07 SiriKit 음성·단축어 연동	미사용	—	쓰지 않는다 — Siri·단축어 진입점이 코드·entitlement 양쪽 모두 0건이다.
08 Nearby Interaction 초광대역 정밀 측위	부분 적용	Core/NearbyExchange/Sources/Ranging/PeerRangingCoordinator.swift	명함 교환 목록에서 상대까지의 거리(예: 2.1m)를 UWB로 실측해 보여준다.
09 WidgetKit 위젯·라이브 액티비티	부분 적용	UMCAppWidget/Sources/UMCAppWidgetBundle.swift	홈 화면 위젯 확장 타겟은 실제로 빌드·탑재되지만, 아직 "준비 중" 안내만 띄운다

이 문서를 만든 방법

- 부스 카드 9장의 원문을 AR 앱 저장소의 `TechCard.swift` 에서 그대로 가져와, 카드 번호와 문서 번호를 일치시켰다.
- 기술마다 조사자를 따로 두어 `UMCApp/` 안의 `import` 지점을 전부 찾고, 거기서부터 `ViewModel` · `UseCase` · `Repository` 를 거꾸로 따라가 사용자 화면까지 연결했다. 동결된 레거시 `AppProduct/` 와 Tuist 생성물 `Derived/` 는 제외했다.
- 같은 기술에 대해 Apple 공식 문서를 받아 정의와 인용문을 확보했다.
- 그 결과를 다른 검증자가 **반대 입장에서** 다시 확인했다 — 인용한 경로가 실재하는지, 줄 번호가 그 심볼을 가리키는지, 코드 조각이 실제 파일과 글자까지 같은지, “핵심” 판정이 과장은 아닌지.
- 검증에서 나온 지적 28건을 본문에 반영했다. 그중에는 Apple 현행 문서에 존재하지 않는 인용문 1건, 프로덕션 코드가 아닌 `#Preview` 안의 줄을 근거로 든 것 1건, 실재와 반대인 서술 1건이 포함된다.

한계

- 조사 시점은 2026년 8월 21일, `feature/1196` 브랜치 기준이다. 코드는 계속 바뀌므로 줄 번호는 시간이 지나면 어긋난다 — 파일 경로와 심볼 이름이 더 오래 간다.
- 정적 분석이다. 코드를 읽어 확인했을 뿐 실제 기기에서 동작을 관찰한 것이 아니다. 특히 “부분 적용” 판정 중 화면까지 닿지 않은 것들은 실행 확인이 아니라 코드 부재로 판단했다.
- 각 장의 “쓰지 않은 것”은 그 기술의 전체 기능을 망라한 목록이 아니라, 이 앱의 맥락에서 눈에 띄는 항목만 골랐다.